

PROYECTO DEL TRABAJO FINAL DE CICLO

Cyber Defense S.L.

Diseño e implementación de una infraestructura de ciberseguridad con honeypot y monitorización centralizada.

Presentado por:

Víctor Rae

Tutora:

Davinia M.

IES Europa

Ciclo Formativo de Grado Superior

Administración de Sistemas Informáticos en Red

Curso académico 2025/2026

Estructura de la memoria

1. Introducción.	3
2. Antecedentes	5
2.1. Situación previa	5
2.2. Soluciones existentes	5
2.3. Marco normativo aplicable	6
3. Objetivos.	8
4. Desarrollo práctico.	9
a) Especificación de requisitos	9
Requisitos funcionales	9
Requisitos no funcionales	10
Pseudo requisitos de sistema y desarrollo	12
b) Análisis y planificación.	15
Infografía	15
Diagrama de Gantt	17
Esquema de red	18
c) Implementación, pruebas y resultados	19
1. Proxmox	19
2. Base de OPNsense	23
3. DMZ	26
3.1. Honeypot	26
3.2. SIEM DMZ	29
4. LAN	36
4.1. FreeIPA Server	36
4.2. SIEM LAN	41
4.3. Estación de trabajo	45
5. Conexión exterior	49
5.1. EC2 - Amazon Web Services	49

5.1.1. Configuración inicial de la instancia	49
5.1.2. Configuración de OPNsense para el túnel WireGuard	52
5.1.3. Configuración de la monitorización en EC2	55
d) Conclusiones	58
e) Nuevas propuestas	62
5. Anexos	64
a) Referencias bibliográficas	64
1. Situación previa	64
2. Implementación	64
b) Glosario de términos	66
c) Índices de tablas, figuras y diagramas	70
d) Instalaciones	72
e) Código fuente	74
1. Modificaciones en Honeypot	74
2. Modificaciones en SIEM DMZ	76
3. Modificaciones en EC2	83

1. Introducción.

La **preocupación por la seguridad digital** de hoy en día es algo **inevitable** y que inquieta cada vez más. **PYMEs y autónomos** son víctimas frecuentes de **ciberataques** que pueden **paralizar su actividad** durante días o incluso semanas. Es por ello que **debemos tomar medidas** aunque no se disponga de los mismos recursos que las grandes corporaciones.

De esta necesidad surge el proyecto. **Cyber Defense S.L.** es una empresa ficticia de **ciberseguridad** que se crea como escenario para este trabajo. Su objetivo ha sido salir al mercado laboral para **concienciar, formar y administrar** la seguridad digital de todo tipo de empresas, desde pequeños negocios hasta grandes empresas.

¿Cómo consigue Cyber Defense ofrecer un buen servicio de seguridad?

La clave está en **conocer al enemigo**. Por eso la empresa cuenta con alta especialización en ciberseguridad y, además, implementa una red muy particular: por un lado, tiene su red interna de trabajo (la de los empleados, los servidores de la empresa, etc.) y, por otro lado, expone a Internet una red con un señuelo o "trampa informática" diseñada específicamente para ser atacada por ciberdelincuentes.

El señuelo simula ser un servidor real pero realmente su única misión es recopilar información técnica sobre los ciberdelincuentes. Toda esta información se envía a un sistema central que la analiza, ordena y muestra en paneles, permitiendo a los analistas de Cyber Defense complementar sus conocimientos sobre los vectores de ataque más utilizados y cómo son explotados en cada momento y, con ese conocimiento, ayudar mejor a sus clientes. El señuelo está preparado para recibir ataques desde Internet ya que para ello se despliega un servidor en la nube y se crea un túnel seguro entre ese servidor y el señuelo alojado en la infraestructura local. De esta forma, cuando un atacante quiera acceder al servidor en la nube, el tráfico se redirige automáticamente al señuelo. Así se consigue exponer el señuelo a internet siguiendo las mismas prácticas de seguridad que utilizan las empresas para proteger sus infraestructuras.

Al final, este proyecto no es solo un montaje técnico, sino una herramienta de inteligencia de amenazas que permite a una empresa real detectar y anticiparse ante los ataques y

proteger mejor sus activos, todo ello utilizando software totalmente libre y sin necesidad de grandes inversiones.

2. Antecedentes

2.1. Situación previa

El problema actual es claro, los ciberataques no dejan de crecer. Según los informes de seguridad que se han recogido en la [bibliografía](#), las PYMEs sufren más del 40% de esos ciberataques. Y lo peor es que muchas de ellas terminan cerrando antes de cumplir seis meses después del golpe. No es solo que los ataques sean cada vez más frecuentes: también hay una falta de concienciación enorme, sobre todo en las pequeñas y medianas empresas, que suelen pensar “a mí no me tocará”.

Y aunque a veces haya concienciación, muchas empresas deciden no hacer nada porque creen que no tienen dinero ni personal técnico. Pero la realidad es otra: con una inversión mínima y herramientas de código abierto se pueden montar sistemas de monitorización y detección que marquen la diferencia entre sufrir un desastre o detectarlo a tiempo.

2.2. Soluciones existentes

Hoy en día hay montones de soluciones de ciberseguridad, tanto comerciales como de código abierto.

Si nos centramos en los honeypots, herramientas como T-Pot (plataforma multi honeypot) o Dionaea (centrado en malware) funcionan muy bien, pero suelen pedir configuraciones bastante más complejas. Trapster, en cambio, ofrece un enfoque más modular y ligero, ideal para lo que buscamos aquí.

En el mundo de los SIEM, opciones comerciales como Splunk o IBM QRadar son muy potentes, pero también muy caras. Alternativas open source como OSSIM o Security Onion existen, pero Wazuh se ha ganado la fama de ser la más completa: integra análisis de logs, detección de vulnerabilidades y monitorización pasiva de archivos.

Para la conexión segura, se optó por un modelo híbrido con una instancia EC2 de AWS y un túnel WireGuard. Es una práctica habitual en entornos profesionales para exponer servicios sin tener que abrir puertos en la red local.

En cuanto al firewall y router principal, opciones comerciales como Palo Alto Networks o Fortinet ofrecen un alto nivel de seguridad, pero con costes de licencia elevados. En el lado de código abierto, pfSense ha sido la referencia durante muchos años. Sin embargo

su interfaz y su modelo de desarrollo se han quedado algo obsoletos para los estándares y facilidades de hoy en día. OPNsense, que se creó como derivado de pfSense, trae una interfaz web, actualizaciones frecuentes y una integración nativa con herramientas como WireGuard, Suricata (IDS/IPS) y Zenarmor (NGFW). Por esta razón se decantó por él, es la opción más completa.

Al final, este proyecto demuestra que es posible construir una infraestructura de ciberseguridad profesional y económica integrando herramientas de código abierto actuales.

2.3. Marco normativo aplicable

El proyecto se alinea con las principales leyes y regulaciones europeas y nacionales en materia de protección de datos y ciberseguridad. Las referencias principales son el **RGPD** y su trasposición nacional mediante la **LOPDGDD**.

Reglamento General de Protección de Datos y Ley Orgánica de Protección de Datos:

El tratamiento de datos sensibles conlleva la obligación de garantizar su confidencialidad, integridad y disponibilidad. El proyecto asume ese mandato como un requisito técnico más, no como un simple trámite documental. Precisamente por la naturaleza crítica de los sistemas monitorizados, se vuelve imprescindible contar con ojos que vigilen la red y sistemas que sepan reaccionar a tiempo; de ahí la lógica de implementar una plataforma robusta de detección y logs.

Alineación con estándares y buenas prácticas:

Más allá del estricto cumplimiento legal, el diseño técnico se apoya en marcos de referencia consolidados para no inventar la rueda. Se ha tenido muy en cuenta el NIST Cybersecurity Framework 2.0, poniendo el foco en dos de sus funciones clave: Detectar (lo que se traduce en nuestra capa de monitorización de amenazas) e Identificar/Proteger (visible en la segmentación de la red y la política de firewalls perimetrales).

Por último, este despliegue no es un fin en sí mismo, sino una base sólida para el futuro. La gestión centralizada de logs y la visibilidad continua de la red son, de hecho, dos de los cimientos que cualquier auditoría sería exigiría para aspirar a un Sistema de Gestión de

Seguridad de la Información (SGSI) bajo el paraguas de la ISO 27001. Con lo que aquí se plantea, la organización daría el primer paso firme hacia esa certificación, resolviendo la parte más técnica y tediosa del proceso.

3. Objetivos.

1. Diseñar una infraestructura de red segmentada, aislando por completo la red interna de producción de la zona desmilitarizada.
2. Seleccionar y justificar el stack de herramientas de software libre más adecuadas para la implementación del entorno de ciberseguridad.
3. Implementar un honeypot en la zona desmilitarizada que simule servicios reales para atraer y registrar ataques de ciberdelincuentes.
4. Desplegar un sistema de monitorización en la zona desmilitarizada, para centralizar y analizar los logs generados por el honeypot.
5. Desplegar un sistema de monitorización en la red interna privada para monitorizar los sistemas y equipos de trabajo de la propia empresa.
6. Configurar dashboards visuales que almacenan y muestran en tiempo real la ejecución, detalles y evolución de ataques
7. Establecer políticas de firewall que garanticen el aislamiento total entre la zona desmilitarizada y la red interna privada, permitiendo únicamente las conexiones necesarias y controladas.
8. Analizar los datos recopilados por el honeypot durante el tiempo de exposición para extraer conclusiones sobre las tendencias actuales de ciberataques.
9. Implementar un modelo de exposición híbrido mediante túnel VPN a una instancia EC2 de AWS que permita recibir ataques reales sin necesidad de abrir puertos en la infraestructura local.

4. Desarrollo práctico.

a) Especificación de requisitos

Partiendo de los objetivos marcados, se ha concretado la siguiente batería de requisitos. Los de corte funcional acotan las tareas que ejecutará la plataforma, mientras que los no funcionales fijan el estándar de calidad y rendimiento exigido. A continuación se recogen también aquellos pseudo requisitos que condicionan el despliegue técnico.

Requisitos funcionales

ID	Requisito funcional
RF-01	Segmentar la red en tres zonas claramente diferenciadas: WAN (salida natural a internet), DMZ (donde residirán los señuelos) y LAN (entorno controlado de clientes internos).
RF-02	Exponer en la DMZ un honeypot que emule un conjunto de servicios atractivos para un atacante, con el fin de registrar en detalle cualquier interacción maliciosa.
RF-03	Centralizar en una instancia SIEM la totalidad de logs que ocupa el honeypot situado en la DMZ, facilitando así las tareas de correlación y análisis posterior.
RF-04	Mantener una segunda instancia SIEM independiente dentro de la LAN. Su misión será vigilar el tráfico de los equipos cliente, preservando la separación lógica con la zona de señuelos.
RF-05	Proporcionar dashboards visuales que reflejen en tiempo real estadísticas de la actividad maliciosa capturada.
RF-06	Aplicar políticas de firewall que impidan cualquier comunicación desde la DMZ a la LAN, garantizando el aislamiento total entre ambas zonas.

RF-07	Posibilitar la exposición controlada del honeypot hacia Internet usando un túnel WireGuard conectado a una instancia EC2 de AWS, sin necesidad de abrir puertos en el router local.
RF-08	Almacenar de forma íntegra los logs generados durante la ventana de exposición, preservando datos clave de los sucesos.
RF-09	Disparar notificaciones automáticas en el momento en que se identifique un posible comportamiento maligno.

Tabla 1: Requisitos funcionales

Requisitos no funcionales

ID	Requisito no funcional	Justificación
RNF-01	Seguridad: Garantizar el aislamiento entre la DMZ y la LAN mediante reglas restrictivas de firewall.	Evitar que un atacante que logre comprometer el honeypot pueda pivotar hacia la red interna de producción.
RNF-02	Disponibilidad: Mantener el entorno operativo 24/7 durante el período de pruebas, con un objetivo de disponibilidad mínima del 99%.	Asegurar que no se pierde ni un solo evento relevante mientras el señuelo permanece expuesto a internet
RNF-03	Rendimiento: Debe ser capaz de manejar al menos 10 conexiones simultáneas al honeypot sin degradación del servicio.	Para simular un escenario realista donde múltiples atacantes pueden intentar acceder a la vez.
RNF-04	Registro (auditoría): Todos los eventos de seguridad deben quedar	Para poder analizar patrones de ataque y generar informes

	registrados con timestamp, IP origen, tipo de evento y resultado.	fiables.
RNF-05	Escalabilidad: La arquitectura debe permitir añadir más equipos en la LAN sin modificar las reglas de firewall principales.	Para demostrar que el diseño es extensible a más clientes en el futuro.
RNF-06	Portabilidad: Todo el software utilizado debe ser libre o de evaluación, permitiendo replicar la infraestructura en otro hardware sin coste de licencia.	Para garantizar que el proyecto es accesible y reproducible.
RNF-07	Usabilidad: Los dashboards deben mostrar la información de forma visual e intuitiva, sin necesidad de formación específica.	Para que los analistas de Cyber Defense puedan interpretar los datos rápidamente.
RNF-08	Mantenibilidad: La configuración del firewall y los servicios debe estar documentada para facilitar futuras modificaciones.	Para que el sistema pueda ser administrado por personal con conocimientos técnicos básicos.

Tabla 2: Requisitos no funcionales

Pseudo requisitos de sistema y desarrollo

El proyecto se desarrolla sobre un único equipo físico que actúa como servidor de virtualización. Las especificaciones de dicho equipo, así como las del hardware que sería necesario en un hipotético despliegue en producción, se detallan a continuación.

Equipo de laboratorio (utilizado para el desarrollo):

Componente	Especificación
Procesador	10 hilos (64 bits) (virtualización) 4.7 GHz
Memoria RAM	26 GB
Almacenamiento	400 GB SSD
Red	Limitado

Tabla 3: Requisitos del laboratorio

Requisitos estimados para un entorno de producción real:

Componente	Mínimo	Recomendado
Procesador	10 núcleos / 20 hilos (64 bits) (virtualización) (+2.0 GHz)	16 núcleos / 32 hilos (64 bits) (virtualización) (+2.7 GHz)
Memoria RAM	64 GB	128 GB
Almacenamiento	2 TB SSD (RAID 1)	4 TB SSD (RAID 10)
Red	1 Gbps	10 Gbps

Tabla 4: Requisitos estimados para producción

Requisitos software

Todo el software utilizado es libre, en versión de evaluación o con licencia disponible, sin coste adicional para el desarrollo del proyecto. Las versiones indicadas son las empleadas durante la implementación.

Software	Versión	Tipo	Función
VMware Workstation Pro	17.6.2	Comercial (con licencia)	Capa de virtualización base
Proxmox VE	9.1	Libre (GPL)	Hipervisor anidado para virtualización
OPNsense	26.1	Libre (BSD)	Firewall y router principal
Wazuh	4.14.3	Libre (GPL)	SIEM (centralización y análisis de logs)
Docker	28.2.2	Libre	Contenerización del honeypot
Trapster	Última	Libre (GPL)	Honeypot (señuelo de trampa informática)
WireGuard	1.0+	Libre (GPL)	Túnel VPN (para conexión con EC2 de AWS)
Debian	13	Libre	Servidores base
Rocky Linux	9	Libre	Servidores base
Linux Mint	22.x	Libre	Estaciones de trabajo

FreeIPA	4.13.1	Libre (GPL)	Gestión centralizada de identidades
----------------	--------	-------------	-------------------------------------

Tabla 5: Requisitos software

Requisitos cloud

Para la exposición segura del honeypot a Internet, se ha utilizado una infraestructura cloud externa que proporciona una IP pública fija y un punto de entrada controlado.

Recurso	Proveedor	Características	Función
Instancia EC2	Amazon Web Services (AWS)	t3.mini	Puerta de entrada pública al honeypot, aloja el túnel WireGuard y redirige el tráfico al honeypot sin necesidad de exponer puertos de la red local a Internet.

Tabla 6: Requisitos cloud

b) Análisis y planificación.

Infografía

En la infografía del proyecto, mostrada en la siguiente página, se representa de forma visual y general la arquitectura completa de Cyber Defense S.L.

Cyber Defense S.L.

Infraestructura de ciberseguridad con honeypot y monitorización centralizada

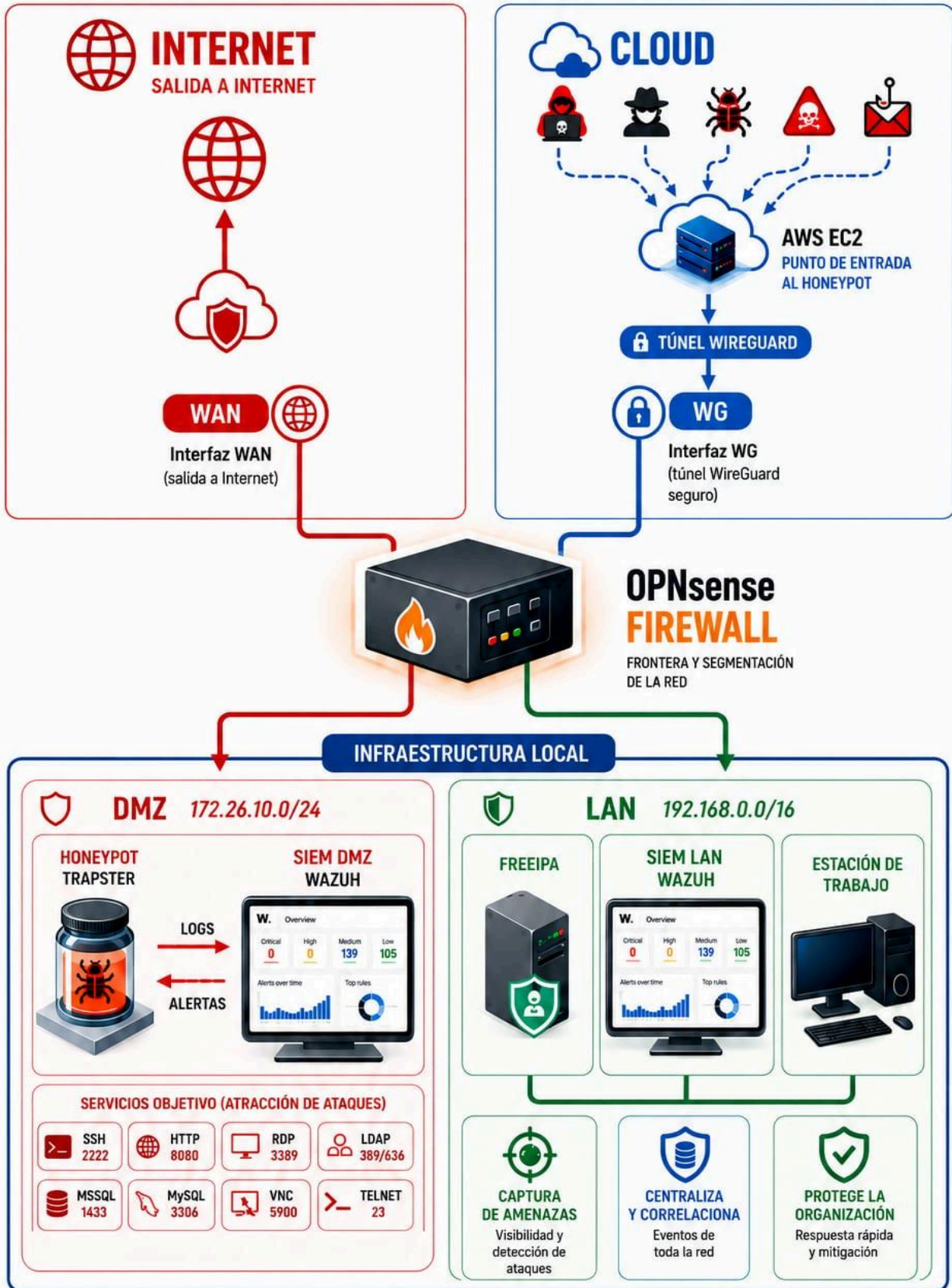


Diagrama de Gantt

El proyecto se desarrolló durante 11 semanas, compatibilizando con el horario de prácticas y otras obligaciones. La planificación se estructuró en fases secuenciales y paralelas para optimizar el tiempo disponible.

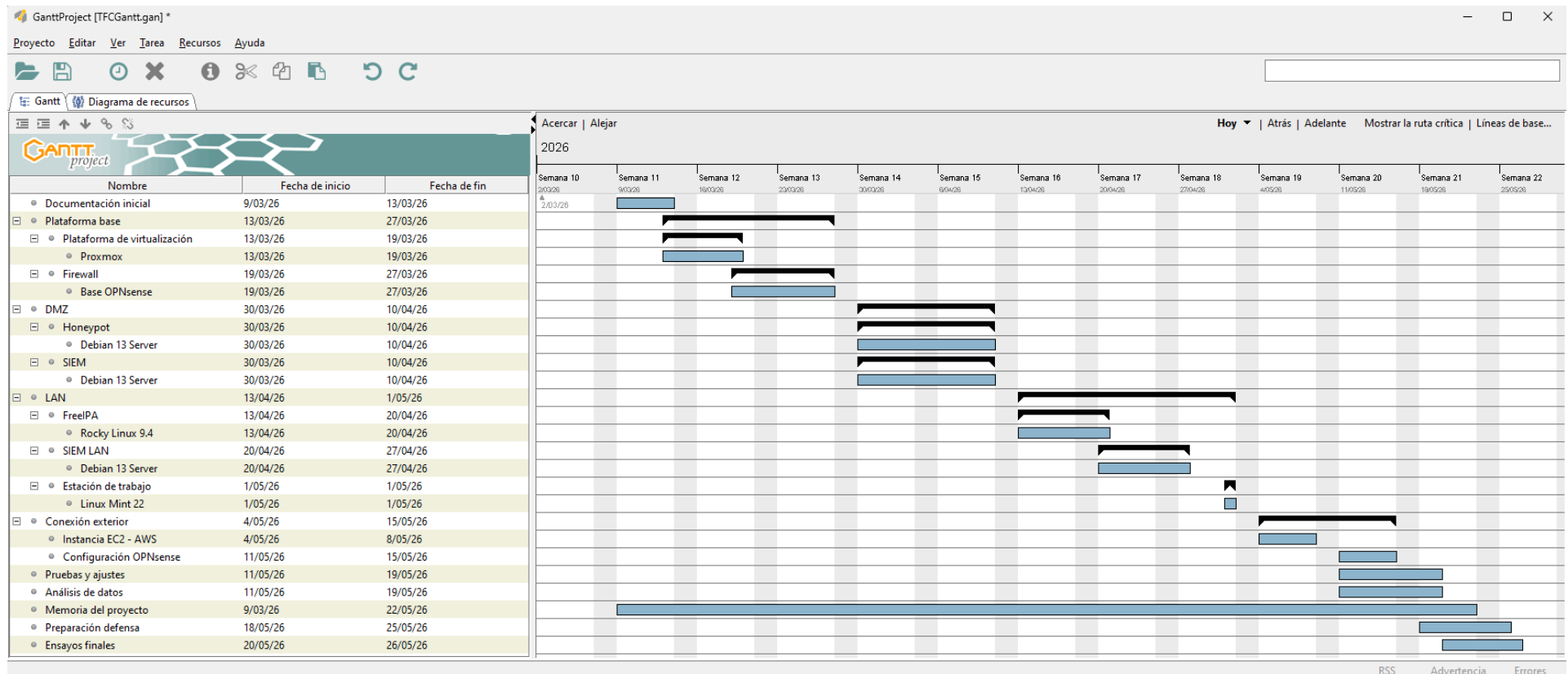


Diagrama 2: Diagrama de Gantt del proyecto

Esquema de red

En la siguiente figura se muestra el esquema de red completo de la infraestructura. Se distinguen tres zonas claramente diferenciadas: la WAN (salida a internet por una parte, y la entrada el honeypot desde la instancia EC2 de AWS), la DMZ (donde se ubican el honeypot y el SIEM DMZ) y la LAN (red interna con FreeIPA, SIEM LAN y equipo de trabajo). Las reglas de firewall garantizan el aislamiento total entre la DMZ y la LAN.

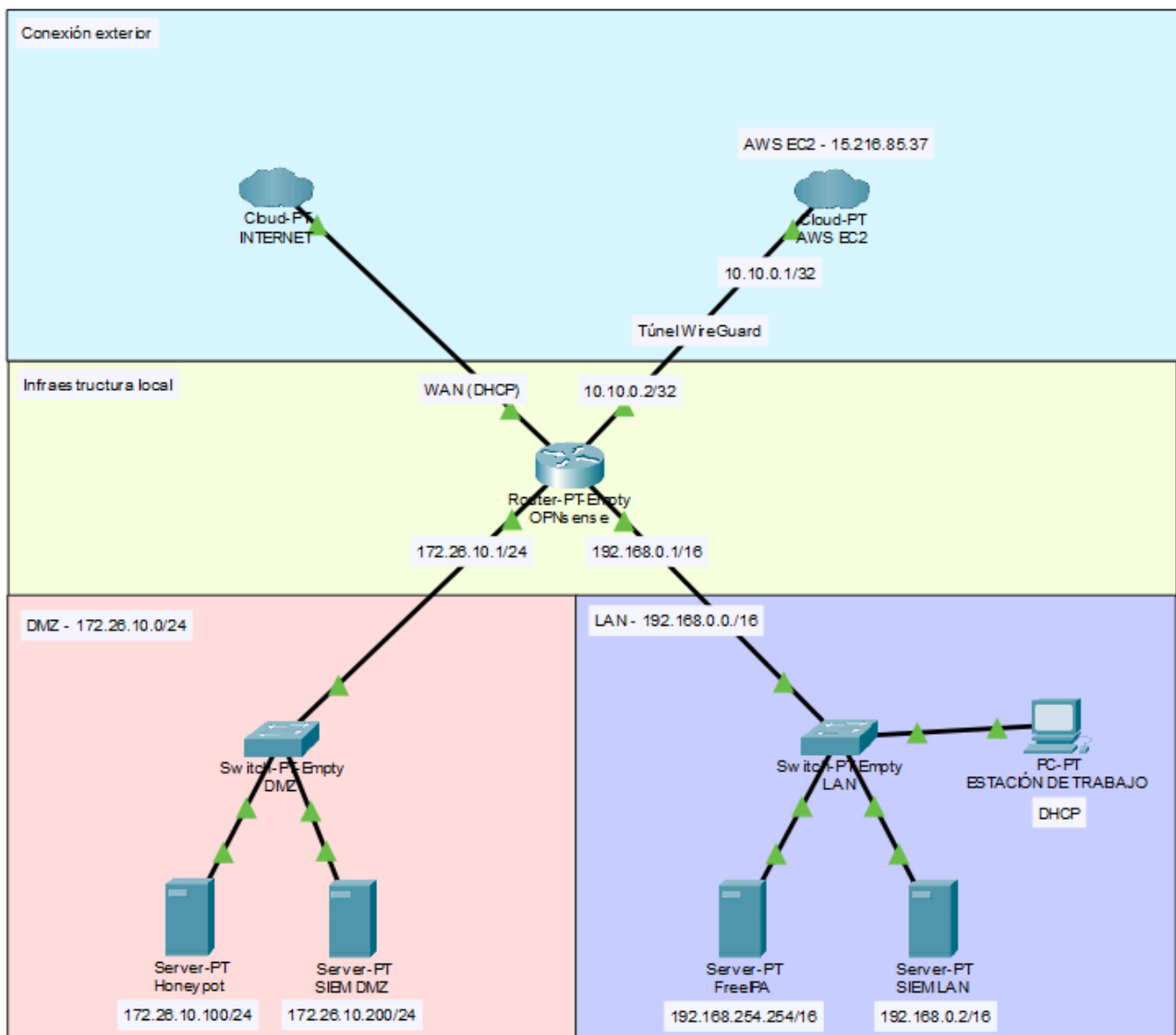


Diagrama 3: Esquema de red de la infraestructura

c) Implementación, pruebas y resultados

1. Proxmox

Para dar soporte a la infraestructura virtualizada, el primer paso fue instalar y configurar Proxmox VE sobre VMware Workstation.

Se empezó configurando la primera plataforma de virtualización (VMWare) para que la máquina dedicada (Proxmox) no tuviese problema en salir a Internet. Para ello y dadas las circunstancias en las que se realizó el proyecto se configuró un adaptador personalizado dentro de VMWare como adaptador puente. Físicamente, el equipo anfitrión estaba conectado a internet mediante un dispositivo Remote NDIS based Internet Sharing Device (móvil con datos compartidos por USB). Con esta configuración, Proxmox quedó en la misma red que el anfitrión, obteniendo una IP del rango correspondiente y pudiendo acceder a internet a través del puente. El resto de adaptadores virtuales se asignaron a redes internas aisladas dentro del propio hipervisor, logrando así la segmentación lógica sin necesidad de múltiples interfaces físicas.

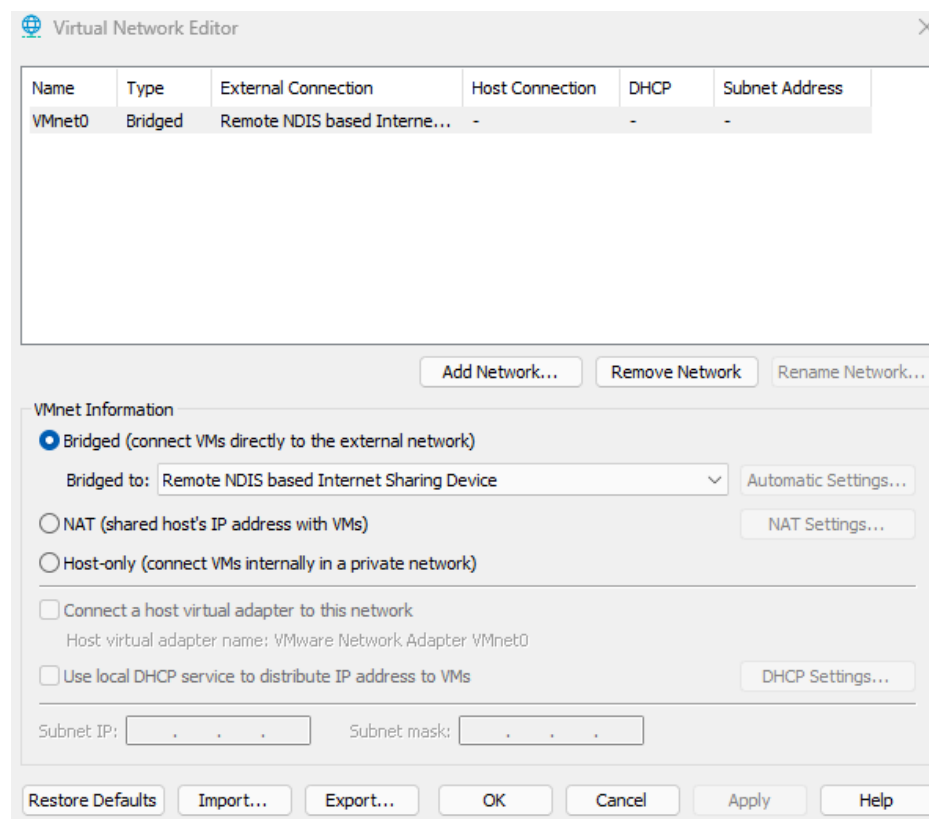


Figura 1: Adaptador de red - VMWare

Por consiguiente lo que se hizo fue crear una máquina virtual con los recursos asignados para toda la infraestructura.

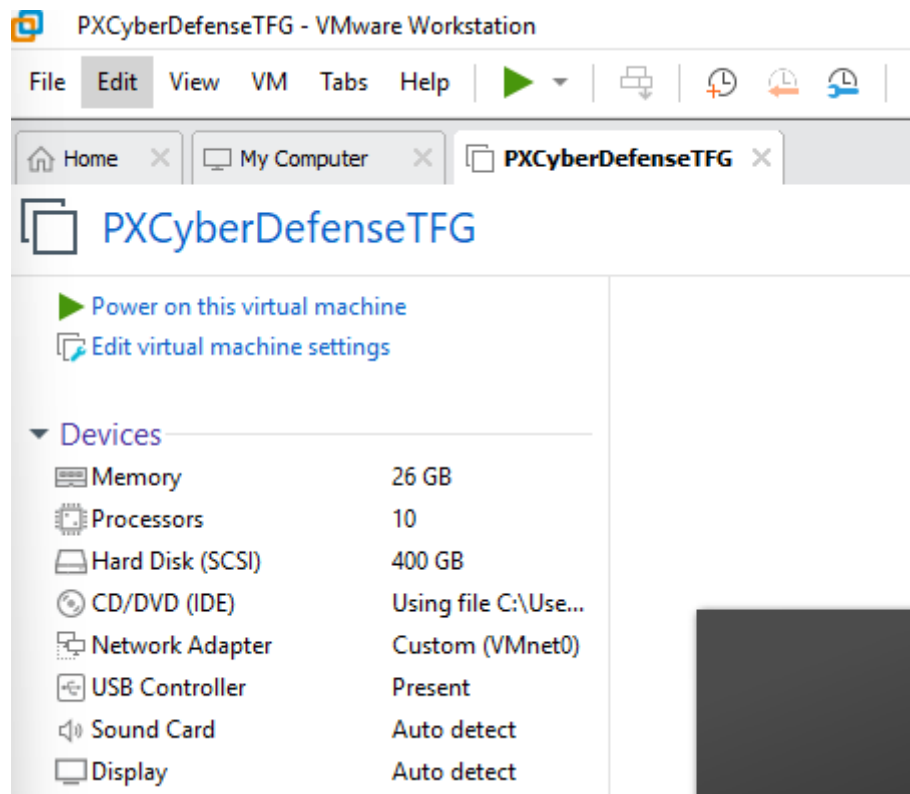


Figura 2: Máquina virtual VMware - Proxmox

La instalación de Proxmox se realizó siguiendo el procedimiento gráfico estándar documentado en la [página web oficial](#). En la **Figura 3** se muestra el resumen final de la instalación, dónde puede verse la configuración de red con IP estática dentro de la subred del anfitrión y el uso de todo el disco asignado en VMWare.



Summary

Please confirm the displayed information. Once you press the **Install** button, the installer will begin to partition your drive(s) and extract the required files.

Option	Value
Filesystem:	ext4
Disk(s):	/dev/sda
Country:	Spain
Timezone:	Europe/Madrid
Keymap:	es
Email:	root@localhost
Management Interface:	nic0
Hostname:	pxcyberdefense
IP CIDR:	10.124.21.1/24
Gateway:	10.124.21.15
DNS:	9.9.9.9

☒ Automatically reboot after successful installation

Abort

Previous

Install

Figura 3: Resumen de la instalación - Proxmox

Tras la instalación, se configuraron los repositorios de Proxmox en modo no-subscription para permitir actualizaciones sin necesidad de licencia, siguiendo la [documentación oficial](#).

Después de esto se configuraron los adaptadores de red virtuales necesarios para la infraestructura.

- **vmbr0:** adaptador principal con salida a internet (WAN de OPNsense).
- **vmbr1:** adaptador para la LAN (sin IP, tráfico interno)
- **vmbr2:** adaptador para la DMZ (sin IP, tráfico aislado)

Para ello se accedió a la interfaz web de Proxmox desde el anfitrión con la dirección IP asignada y por el puerto correspondiente (**10.124.21.1:8006**), autenticándose con el usuario root del sistema (autenticación PAM estándar).

La configuración de estos adaptadores se realizó desde el nodo **pxcyberdefense** → **System** → **Network** → **Create**.

El resultado final puede verse en la **Figura 4**.

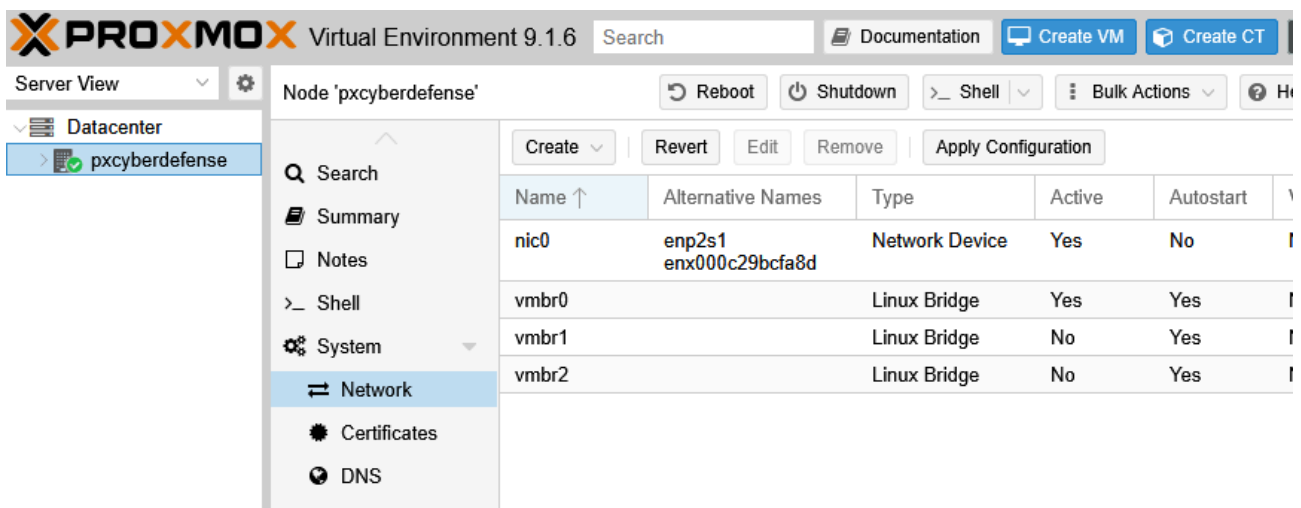


Figura 4: Adaptadores de red - Proxmox

Además, se procedió a crear un grupo y usuario asociado como administrador para usarse como administración en la interfaz web de proxmox y así evitar el uso de root.

Llegados a este punto se subieron las imágenes ISO necesarias para las máquinas virtuales del proyecto. Para ello, desde la interfaz web de Proxmox se accedió a la sección **pxcyberdefense** → **local** → **ISO Images**, y se utilizó la opción de subida de archivos. En la **Figura 5** se muestra el listado de ISOs disponibles tras la subida.

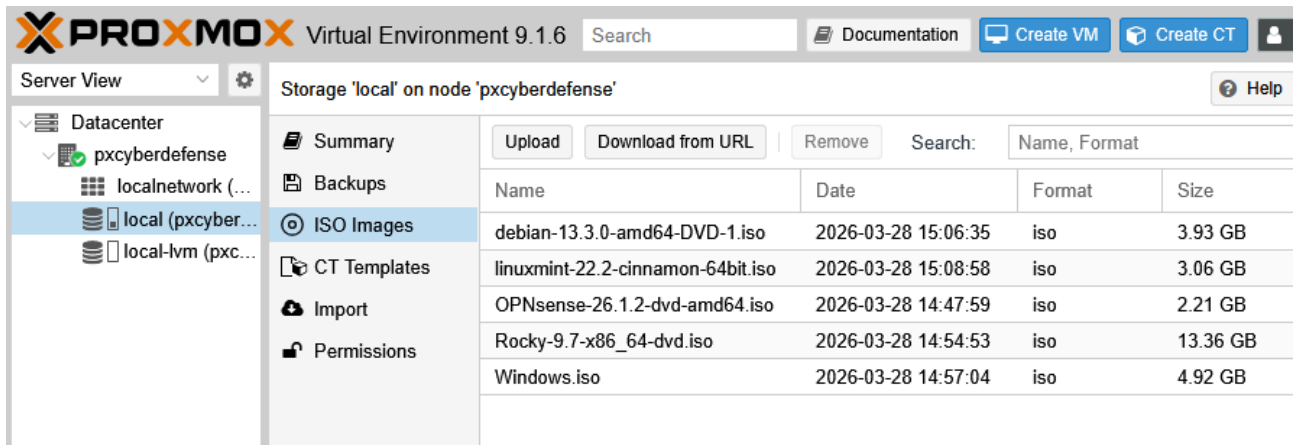


Figura 5: Lista de ISO's subidas - Proxmox

Una vez subidas las ISO's y teniendo configurado Proxmox, se instaló la base y firewall del sistema, OPNsense.

2. Base de OPNsense

Para la instalación y configuración de OPNsense se creó una máquina virtual exclusiva dentro de Proxmox.

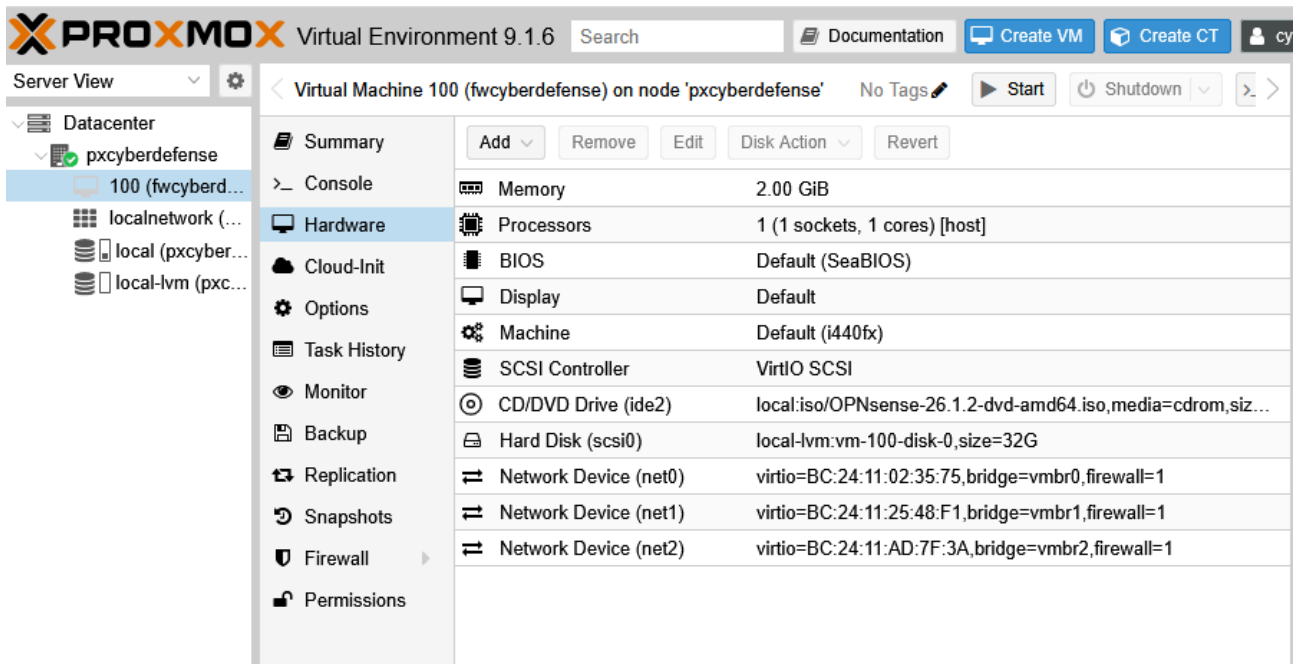
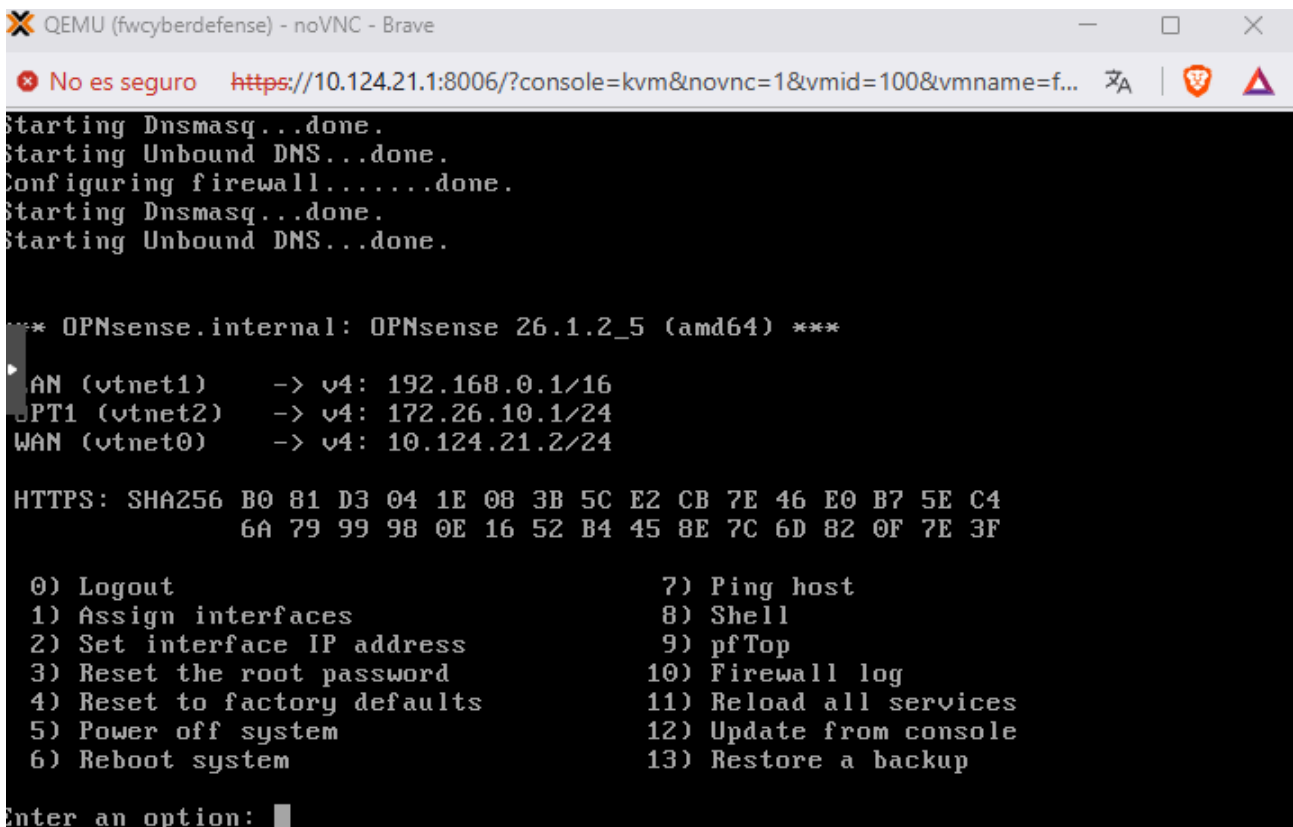


Figura 6: Máquina virtual - OPNsense

Se siguió el procedimiento estándar del [manual de instalación oficial](#) para instalar OPNsense.

Tras la instalación las interfaces quedaron configuradas de tal manera, donde LAN es la red privada (192.168.0.1/16), OPT1 es la zona desmilitarizada (172.26.10.1/24) y WAN es la salida a Internet que comparte red con el anfitrión.



```
QEMU (fwcyberdefense) - noVNC - Brave
No es seguro https://10.124.21.1:8006/?console=kvm&novnc=1&vmid=100&vmname=f...
Starting Dnsmasq...done.
Starting Unbound DNS...done.
Configuring firewall.....done.
Starting Dnsmasq...done.
Starting Unbound DNS...done.

*** OPNsense.internal: OPNsense 26.1.2_5 (amd64) ***

LAN (vtnet1)    -> v4: 192.168.0.1/16
OPT1 (vtnet2)   -> v4: 172.26.10.1/24
WAN (vtnet0)    -> v4: 10.124.21.2/24

HTTPS: SHA256 B0 81 D3 04 1E 08 3B 5C E2 CB 7E 46 E0 B7 5E C4
              6A 79 99 98 0E 16 52 B4 45 8E 7C 6D 82 0F 7E 3F

0) Logout                      7) Ping host
1) Assign interfaces           8) Shell
2) Set interface IP address    9) pfTop
3) Reset the root password     10) Firewall log
4) Reset to factory defaults   11) Reload all services
5) Power off system            12) Update from console
6) Reboot system               13) Restore a backup

Enter an option: 
```

Figura 7: Interfaces en terminal - OPNsense

Teniendo configuradas las interfaces, se debe saber que por seguridad la interfaz web de OPNsense sólo escucha en la red privada. Por lo tanto, para acceder a ella es necesario tener un dispositivo dentro de esa red.

Una vez configurado el proceso inicial en la web las interfaces quedan así:

The screenshot shows the OPNsense web interface. The top navigation bar includes the OPNsense logo, the text 'root@OPNsense.internal', and a search bar. The left sidebar contains a menu with options: Reporting, System, Interfaces, [DMZ], [LAN], [WAN], Assignments, Devices, Neighbors, Overview (selected), Settings, Virtual IPs, Wireless, and Diagnostics. The main content area is titled 'Interfaces: Overview' and contains a table with the following data:

Status	Interface	Device	VLAN	Link Type	IPv4	IPv6	Gateway	Routes
🟢	WAN (wan)	vtnet0		static	10.124.21.2/24	fe80::be24:11ff:fe	10.124.21.15	default 10.124.21.0/24 Expand
🟢	LAN (lan)	vtnet1		static	192.168.0.1/16	fe80::be24:11ff:fe		192.168.0.0/16 fe80:: %vtnet1/64
🟢	DMZ (opt1)	vtnet2		static	172.26.10.1/24			172.26.10.0/24
🟢	Loopback (lo0)	lo0		static	127.0.0.1/8	::1/128 fe80::1/64		10.124.21.2 127.0.0.1 Expand

Figura 8: Interfaces en la web - OPNsense

Una vez configuradas las interfaces, se actualizó el equipo siguiendo la [documentación oficial](#) de OPNsense.

Tras la actualización, se configuraron las reglas del firewall para la zona desmilitarizada (DMZ), ya que por defecto está totalmente aislada. Para ello, en **Firewall** → **Rules** → **DMZ** se crearon dos reglas en el siguiente orden:

- 1ª regla → **Bloquea** el tráfico dirigido a la LAN.
- 2ª regla → **Permite** el tráfico a todo.

El orden de las reglas es muy importante ya que al haber posicionado la regla del bloqueo la primera, no se permitirá la comunicación a la LAN, de lo contrario aunque la regla estuviese activa pero en segundo lugar no surte efecto ya que la solaparía el permitir todo, la configuración final de estas reglas se puede ver en la **Figura 9**.

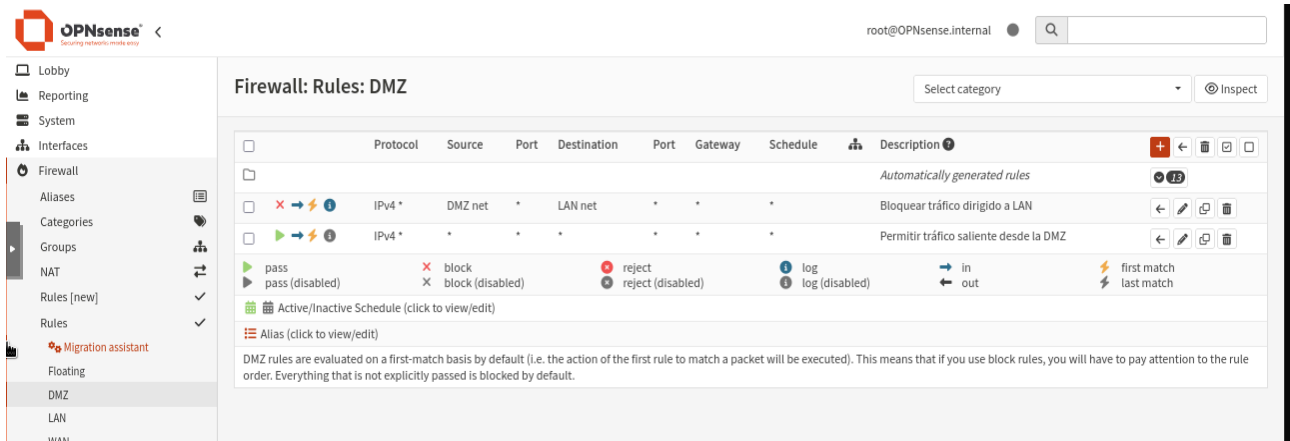


Figura 9: Reglas DMZ - OPNsense

3. DMZ

3.1. Honeypot

Para la instalación y configuración del honeypot se creó una máquina virtual dentro de Proxmox, con Debian 13 Server, en la **Figura 10** se muestran sus especificaciones.

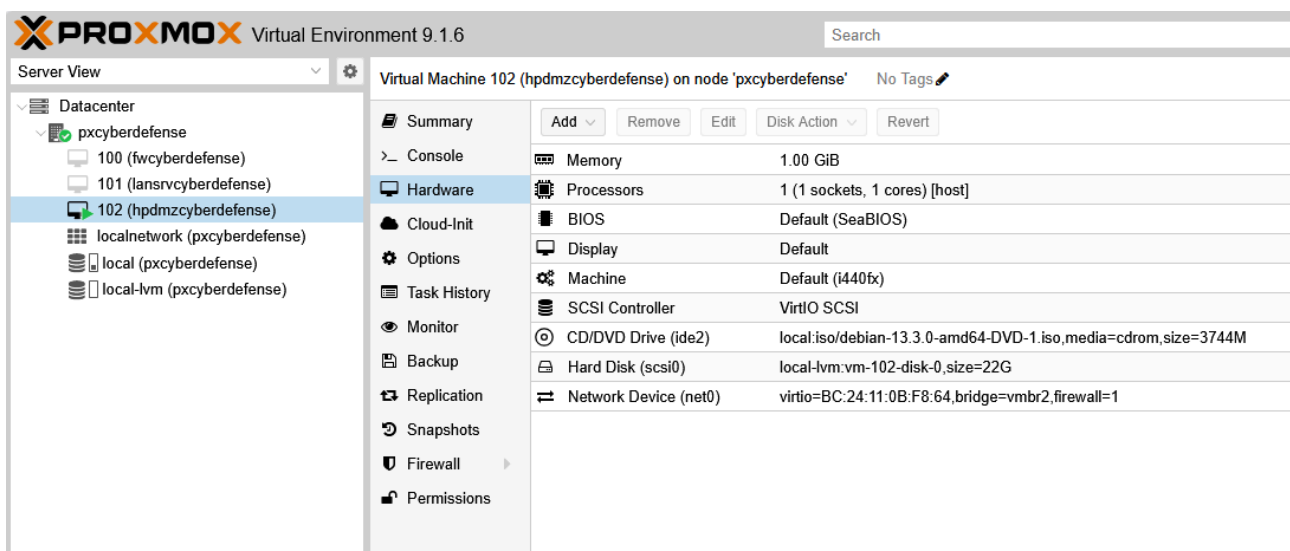


Figura 10: Máquina virtual - Honeypot

Se siguió el procedimiento estándar del [manual de instalación oficial](#) para instalar Debian 13.

Una vez instalado el sistema, se configuró la interfaz de red y se asignó la dirección IP estática 172.26.10.100/24, con lo que la máquina quedó dentro de la zona desmilitarizada, tal como se muestra en la **Figura 11**.

```
cyberad@hpdmczyberdefense:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether bc:24:11:0b:f8:64 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24110bf864
    inet 172.26.10.100/24 brd 172.26.10.255 scope global ens18
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe0b:f864/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
cyberad@hpdmczyberdefense:~$ ip r
default via 172.26.10.1 dev ens18 onlink
172.26.10.0/24 dev ens18 proto kernel scope link src 172.26.10.100
cyberad@hpdmczyberdefense:~$
```

Figura 11: Interfaz de red - Honeypot

A continuación se configuraron los repositorios oficiales de Debian y se actualizó el sistema siguiendo la [documentación oficial](#) de la página web de Debian.

Para la instalación y despliegue del honeypot Trapster Community, se siguió el proceso de despliegue respectivo con Docker, mencionado en el [repositorio oficial](#).

Con la máquina lista, se instaló Docker y una de sus herramientas con el comando **sudo apt install docker.io docker-compose -y**.

Tras la instalación de Docker, se clonó el repositorio oficial de Trapster desde GitHub. Para adaptarlo al proyecto, se realizaron las siguientes modificaciones:

- Se editó el archivo `trapster-community/trapster/data/trapster.conf` para cambiar el sistema de logs, sustituyendo el logger por defecto por un **FileLogger** que escribiera los eventos en un archivo, esto se hizo siguiendo la siguiente [documentación](#). Para que estos logs persistieran y fueran accesibles desde el exterior del contenedor, se modificó también el archivo `trapster-community/docker-compose.yml` montando un volumen que mapeaba la carpeta **logs** del repositorio con la ruta donde Trapster generaba los archivos de log dentro del contenedor. El detalle de estos cambios puede consultarse en el [Anexo E](#).
- Se modificó el archivo `logger.py` para que las marcas de tiempo se generasen en formato ISO 8601, ya que es un requisito indispensable para que Wazuh pudiera interpretar correctamente los eventos, [Anexo E](#).
- Se personalizó la plantilla de la página web servida por el puerto 8080 (http) por el honeypot (carpeta `trapster-community/trapster/data/http/demo_api`) con ayuda de herramientas de inteligencia artificial, mejorando la apariencia y el realismo, para realizar este cambio también se tuvo que modificar el archivo `docker-compose.yml`, para que el contenedor montase y usase la carpeta local del repositorio con las páginas web modificadas, véase en el [Anexo E](#).

Una vez realizados estos ajustes, se desplegó el contenedor en segundo plano, en la **Figura 12** se muestra el contenedor en ejecución.

```
cyberad@hpdmczyberdefense:~/trapster-community$ sudo docker compose up -d
[+] Running 1/1
  ⬆ Container trapster-community Started
cyberad@hpdmczyberdefense:~/trapster-community$ sudo docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES                IMAGE                                STATUS
trapster-community   trapster-community-trapster-community Up 6 seconds
cyberad@hpdmczyberdefense:~/trapster-community$ _
```

Figura 12: Contenedor de Trapster - Honeypot

3.2. SIEM DMZ

Para la instalación y configuración del SIEM Wazuh, se creó una máquina virtual dentro de Proxmox con las especificaciones que se muestran en la **Figura 13**.

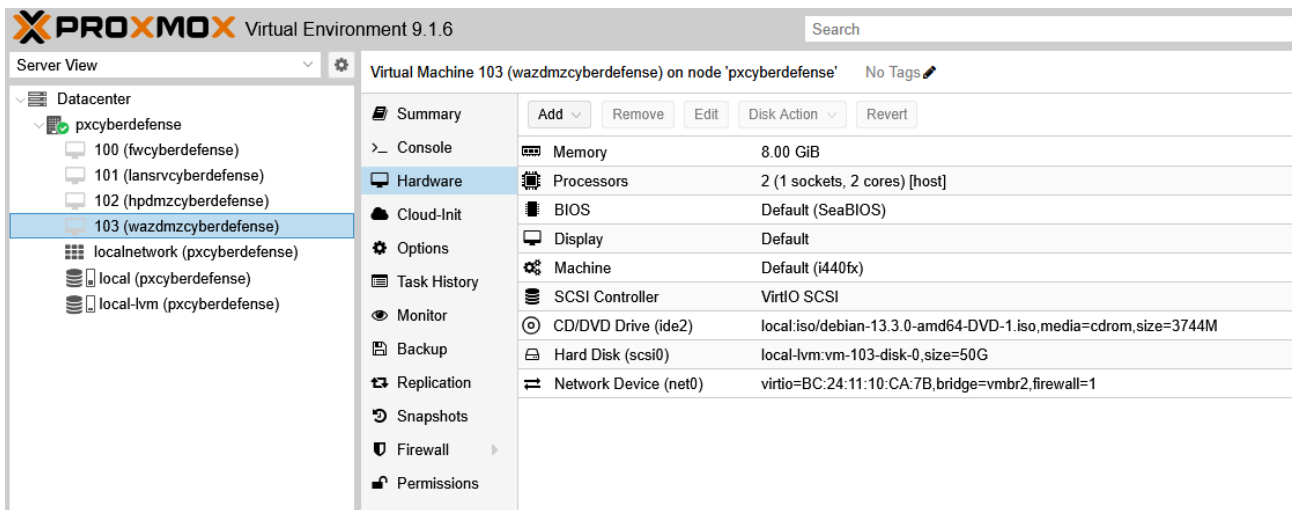


Figura 13: Máquina virtual - SIEM DMZ

Para la máquina destinada al SIEM, se aplicó el mismo procedimiento de instalación y actualización de Debian 13 utilizado en el Honeypot.

Tras la instalación del sistema operativo, se configuró la interfaz de red dentro de la zona desmilitarizada y se le asignó a la máquina la dirección IP estática **172.26.10.200/24**, como se muestra en la **Figura 14**.

```

cyberad@wazdmzcyberdefense:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:10:ca:7b brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc241110ca7b
    inet 172.26.10.200/24 brd 172.26.10.255 scope global ens18
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe10:ca7b/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
cyberad@wazdmzcyberdefense:~$ ip r
default via 172.26.10.1 dev ens18 onlink
172.26.10.0/24 dev ens18 proto kernel scope link src 172.26.10.200
cyberad@wazdmzcyberdefense:~$

```

Figura 14: Interfaz de red - SIEM DMZ

Con la máquina preparada, se procedió a instalar Wazuh mediante el modo todo en uno (wazuh-manager, indexer & dashboard) siguiendo el [manual oficial de instalación](#). Para ello se descargó y ejecutó el script desde los [repositorios oficiales](#).

Los 3 componentes de **Wazuh** tal y como se han mencionado, junto a su función son:

Wazuh Manager: Actúa como el núcleo del sistema. Recibe los eventos enviados por los agentes, los analiza mediante reglas de seguridad y genera alertas en cuanto detecta actividad sospechosa.

Wazuh Indexer: Funciona como el motor de búsqueda y almacenamiento. Almacena los eventos y las alertas de forma estructurada, lo que permite consultar y recuperar la información con rapidez y eficiencia.

Wazuh Dashboard: Constituye la interfaz gráfica web. Presenta los datos guardados por el indexador en paneles visuales, facilitando así la monitorización, el análisis de alertas y la gestión general del sistema.

Una vez finalizada la instalación se accedió al dashboard de Wazuh a través de la url **https://172.26.10.200**. En la **Figura 15** se muestra la pantalla de inicio de sesión del dashboard.

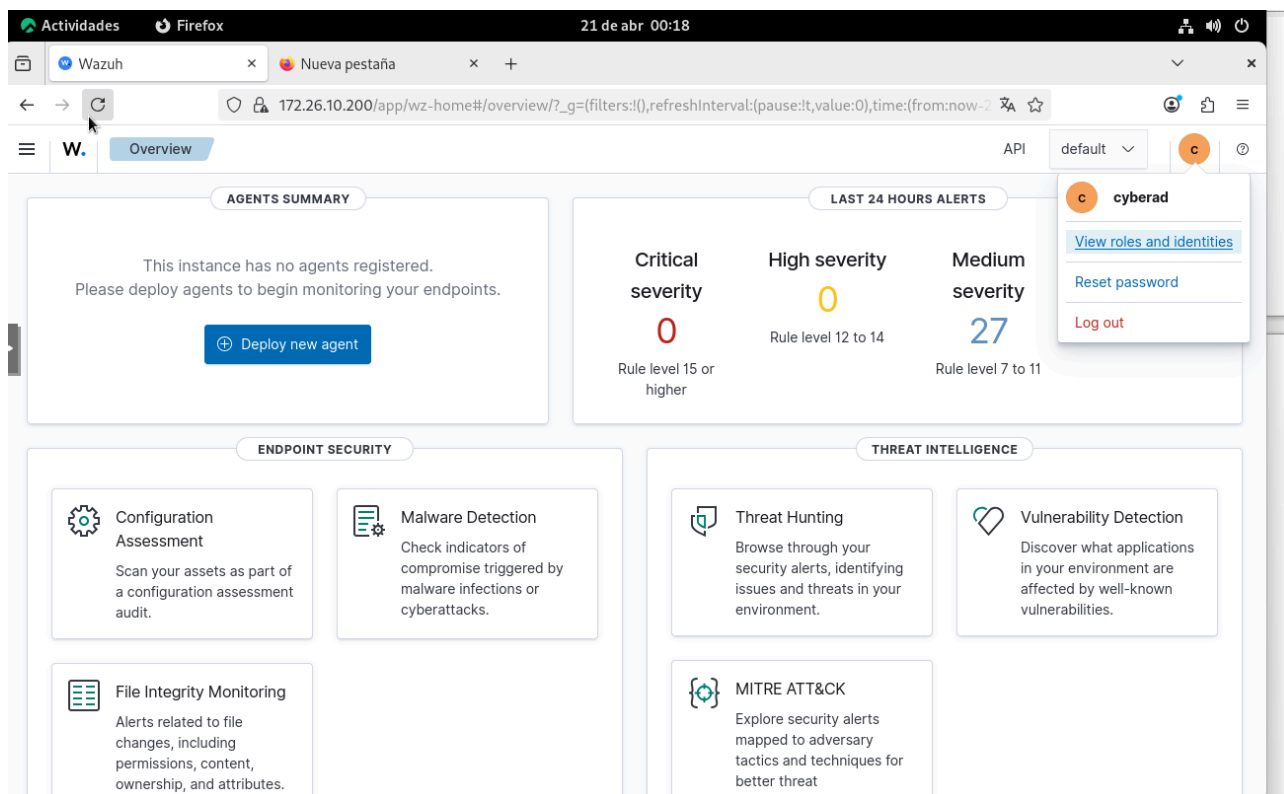


Figura 15: Wazuh Dashboard - SIEM DMZ

Una vez operativo el SIEM, se registró como nuevo agente al honeypot para que los logs fuesen recibidos y analizados por Wazuh. Para ello, desde el dashboard se generó el comando correspondiente desde **Desplegar nuevo agente** y se ejecutó en el agente. Una vez registrado, el agente aparece en la lista de agentes tal y como se muestra en la **Figura 16**.

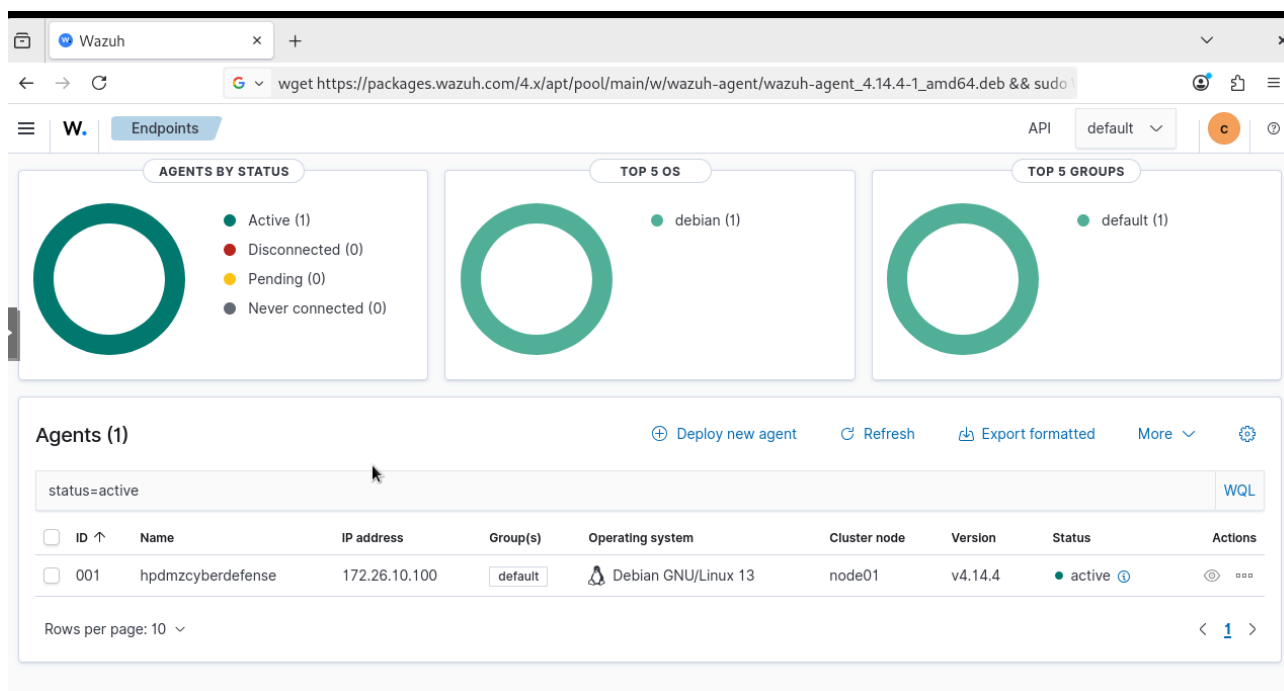


Figura 16: Agente registrado en honeypot - SIEM DMZ

Tras haber registrado al agente, se tuvo que configurar para que se pudiera recopilar los logs generados por el contenedor del honeypot, para ello se editó el archivo de configuración del agente de Wazuh añadiendo un bloque **localfile** para apuntar a la ruta donde Trapster almacena sus logs.

Además, para que Wazuh generase alertas específicas sobre los eventos del honeypot, se definieron reglas personalizadas en el servidor de Wazuh. Estas reglas cubren servicios como SSH, FTP, RDP, HTTP, HTTPS, Telnet, LDAP, MySQL, MSSQL, VNC, DNS y Rsync. Todas estas configuraciones se pueden consultar en el [Anexo E](#).

En la **Figura 17** se muestran varias alertas reales tras la operatividad de las reglas.

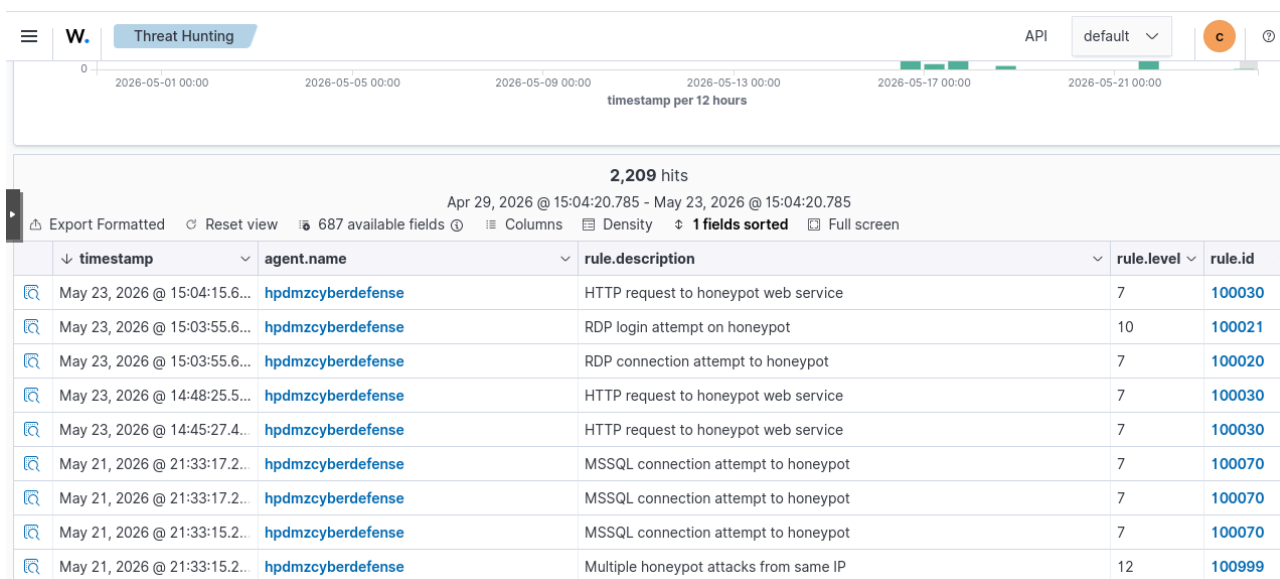


Figura 17: Alertas HoneyPot - SIEM DMZ

Las reglas se han organizado en dos grandes grupos según la naturaleza del evento detectado en el honeypot:

1. Conexiones a servicios (nivel 7)

Se activan cuando un atacante **establece** una **conexión** con cualquiera de los servicios simulados: SSH, FTP, RDP, HTTP, HTTPS, Telnet, LDAP, MySQL, MSSQL, VNC, DNS y Rsync. Estas reglas utilizan el campo logtype proporcionado por Trapster para identificar el servicio concreto. Su nivel 7 permite alertar sin generar ruido excesivo. Se han mapeado a las técnicas MITRE T1595 (Active Scanning) y T1046 (Network Service Discovery).

2. Intentos de autenticación (nivel 10)

Se activan cuando se detecta un **intento** de **login** (ej. ssh.login, ftp.login, rdp.login, etc.). Indican que el atacante ha pasado del escaneo a la fase de prueba de credenciales. Se han clasificado con T1021 (Remote Services) y T1110 (Brute Force). Para consultas LDAP se ha utilizado un nivel 8 y las técnicas T1087 (Account Discovery) y T1018 (Remote System Discovery).

3. Fuerza bruta global (nivel 12)

La regla 100999 actúa como un umbral general: si una misma IP supera **5 eventos** de cualquier tipo del honeypot en **10 segundos**, se eleva la alerta a nivel 12. Esta regla detecta comportamientos de fuerza bruta o escaneo intensivo independientemente del servicio atacado.

Tras recibir las alertas con las IP's públicas después de configurar la conexión exterior mostrada en el [punto 5](#), se procedió a configurar un **dashboard personalizado** dentro del SIEM con el fin de mostrar la evolución de los ataques a la infraestructura.

Para ello se accedió a la parte superior izquierda del dashboard de Wazuh, que muestra un menú desplegable, y se accedió concretamente a **Explore → Dashboards**. Dentro, se creó un nuevo dashboard, al que se le añadieron tres visualizaciones: una tabla de datos, un mapa de coordenadas y un gráfico de barras.

Para la **tabla de datos** se utilizó el campo **data.srcip** como término de agregación, ordenando de **forma descendente** por el número de eventos. De esta manera se obtiene un listado de las IPs atacantes más frecuentes. En el **mapa de coordenadas** se empleó el campo **GeoLocation.location**, que **Wazuh genera automáticamente** a partir de la IP de origen cuando la geolocalización está activada, permitiendo visualizar la procedencia geográfica de los ataques. Finalmente, para el gráfico de barras, se aplicó una agregación **Date Histogram** en el eje X. Se configuró para que agrupara los eventos utilizando el campo **timestamp** como referencia temporal para agrupar los eventos.

El conjunto de las visualizaciones (dashboard) se guardó bajo el nombre **Attack Monitor**.

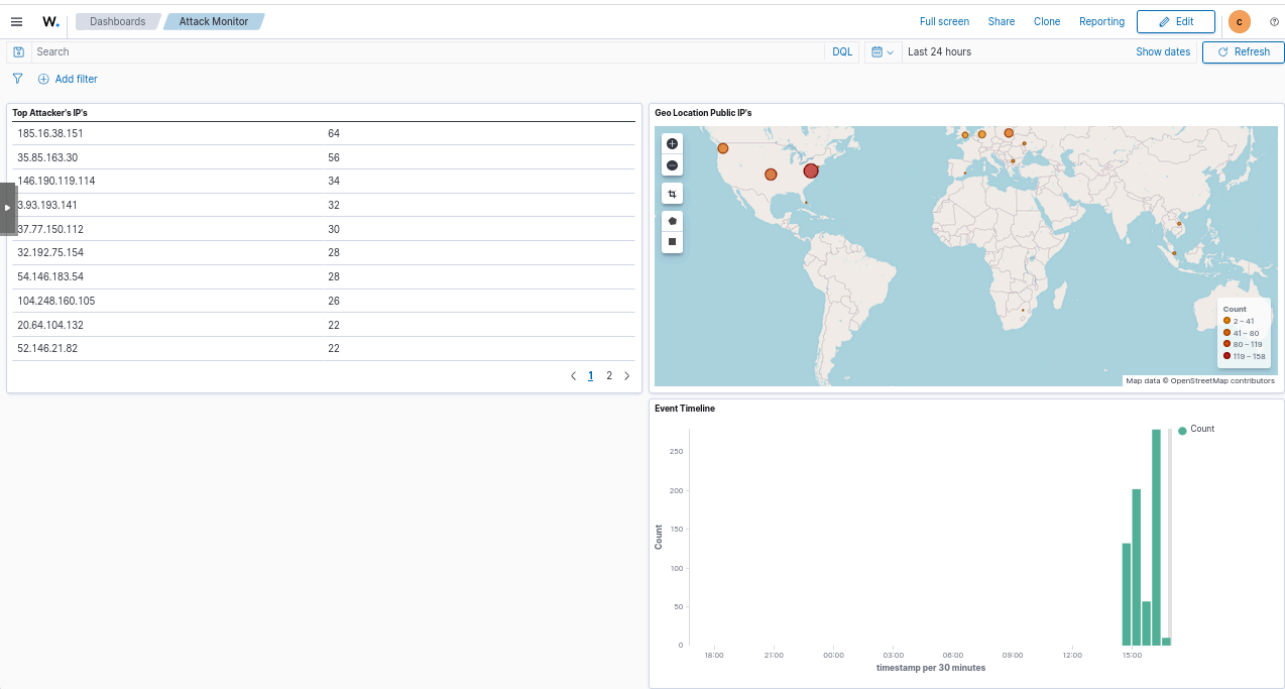


Figura 18: Dashboard personalizado - SIEM DMZ

4. LAN

4.1. FreeIPA Server

Inicialmente se barajó la posibilidad de alojar FreeIPA y Wazuh LAN en un mismo servidor para optimizar recursos, pero tras analizar las implicaciones de seguridad y operación se descartó esta opción. Ambos servicios requieren el puerto 443 para sus respectivas interfaces web, lo que provoca un conflicto directo. Además, separar el servicio crítico de autenticación (FreeIPA) del sistema de monitorización (Wazuh) reduce la superficie de ataque y evita que una vulnerabilidad en uno pueda comprometer al otro, siguiendo el principio de segmentación de servicios. Por último, mantenerlos en máquinas virtuales independientes facilita la administración, la resolución de problemas y la asignación de recursos específicos a cada uno. Por ello, se decidió desplegar FreeIPA en su propia VM y el servidor Wazuh LAN en otra máquina virtual independiente.

Se empezó la red interna con el servidor para la gestión centralizada de usuarios, políticas de acceso y autenticación en el dominio de Cyber Defense S.L., implementando para ello un servidor FreeIPA sobre una máquina virtual con Rocky Linux 9.7, cuyas especificaciones se muestran en la **Figura 19**.

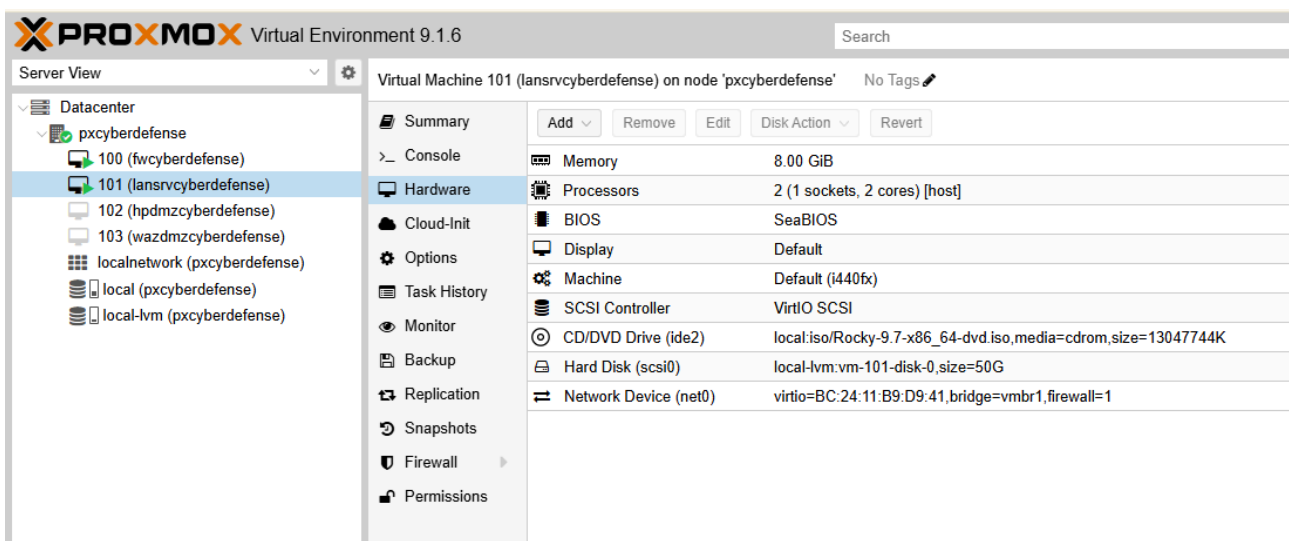


Figura 19: Máquina virtual - FreeIPA

Se instaló el sistema base de Rocky Linux siguiendo el procedimiento estándar documentado en la [página oficial](#).

Una vez instalado el sistema y antes de proceder con la instalación de FreeIPA, se reservó en OPNsense una dirección estática para el servidor, en la **Figura 20** se puede ver la dirección asignada.

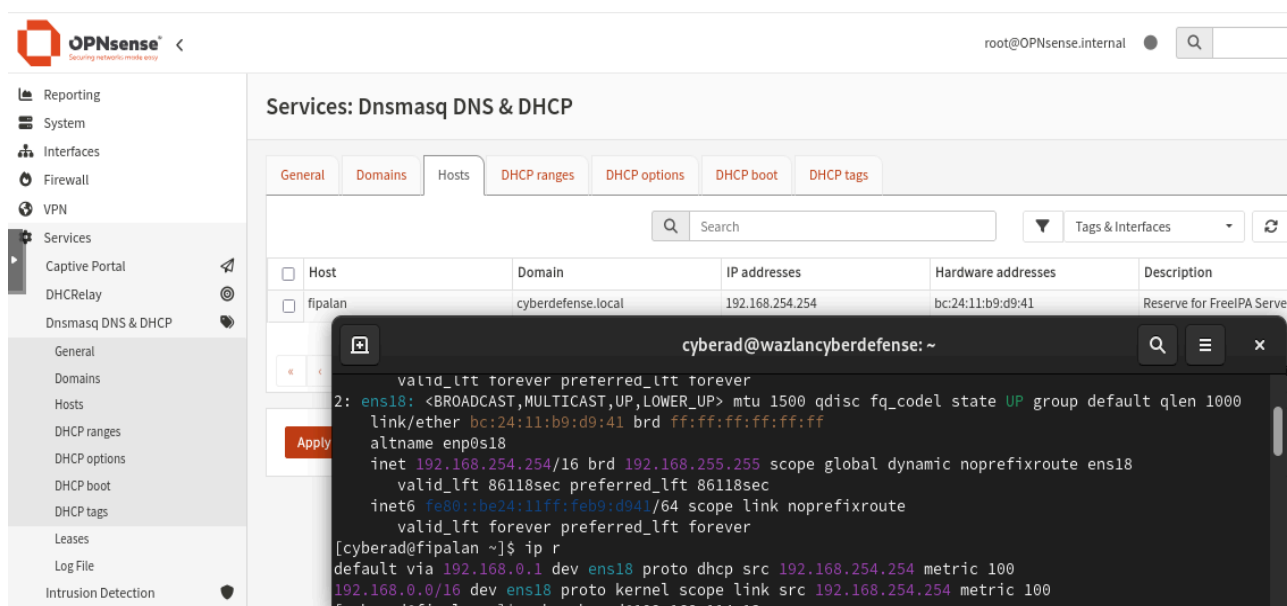


Figura 20: Reserva DHCP - FreeIPA

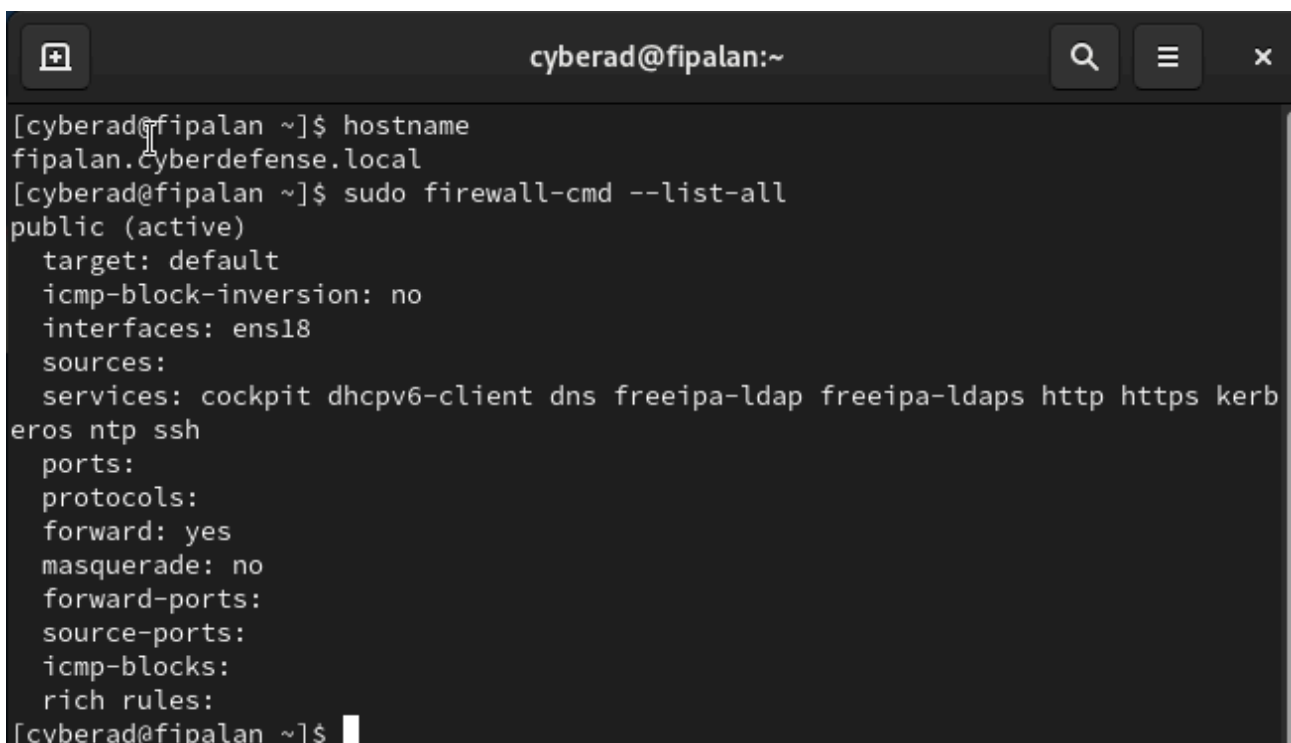
Tras haber configurado la red se procedió a actualizar los repositorios y sistema a la última versión siguiendo la [documentación oficial](#) ubicada en la web de Rocky Linux.

A continuación se preparó la máquina para empezar a instalar FreeIPA siguiendo la [guía de instalación oficial](#).

Como pasos previos a la instalación de FreeIPA se hizo lo siguiente:

- Se configuró un FQDN (Fully Qualified Domain Name) o nombre de dominio completo para la máquina, ya que se requiere de ello, también se modificó la línea respectiva en **/etc/hosts** asegurando que no hubiese inconsistencias ni en las direcciones IP ni en los nombres.
- Se abrieron los puertos para los servicios respectivos (LDAP/s,DNS,NTP,HTTP/S,Kerberos) en el firewall de FreeIPA.

Se puede ver el resultado de estas en la **Figura 21**.



```

cyberad@fipalan:~
[cyberad@fipalan ~]$ hostname
fipalan.cyberdefense.local
[cyberad@fipalan ~]$ sudo firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens18
  sources:
  services: cockpit dhcpv6-client dns freeipa-ldap freeipa-ldaps http https kerberos ntp ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[cyberad@fipalan ~]$

```

Figura 21: Pre-requisitos - FreeIPA

Cumplidos los requisitos previos, se instalaron los paquetes necesarios para el servidor FreeIPA y su servidor DNS integrado.

Esto se realizó con el comando **\$ sudo dnf install freeipa-server freeipa-server-dns -y**.

A continuación se lanzó el asistente de configuración con la opción para habilitar el servidor DNS interno, con el comando: **\$ sudo ipa-server-install -setup-dns**.

Durante el proceso asistido se definieron los siguientes parámetros:

- **Dominio:** cyberdefense.local
- **Realm:** CYBERDEFENSE.LOCAL
- **Contraseña** para el administrador del directorio (Directory Manager) y para el usuario administrador de FreeIPA (admin).

Se aceptó la configuración propuesta para los reenviadores DNS (forwarders), utilizando los servidores DNS del gateway (OPNsense). También se habilitó la creación de zonas inversas.

El instalador completó todas las tareas de configuración sin errores, generando la infraestructura necesaria (servidor LDAP, Kerberos KDC, DNS, certificados, etc.). Una vez finalizado el proceso, se verificó el estado de los servicios con el comando **\$ sudo ipactl status**.

En la **Figura 22** se muestra el resultado de esta verificación, donde puede observarse que todos los componentes se encuentran activos y en ejecución.


```
cyberad@fipalan:~  
[cyberad@fipalan ~]$ sudo ipactl status  
[sudo] password for cyberad:  
Directory Service: RUNNING  
krb5kdc Service: RUNNING  
kadmin Service: RUNNING  
named Service: RUNNING  
httpd Service: RUNNING  
ipa-custodia Service: RUNNING  
pki-tomcatd Service: RUNNING  
ipa-otpd Service: RUNNING  
ipa-dnskeysyncd Service: RUNNING  
ipa: INFO: The ipactl command was successful  
[cyberad@fipalan ~]$
```

Figura 22: Estado de los servicios de FreeIPA

Para acceder a la interfaz gráfica de administración, mediante un navegador, se introdujo la URL **https://192.168.254.254** (o **https://fipalan.cyberdefense.local**). Tras aceptar el certificado autofirmado, se presentó la pantalla de inicio de sesión. Con las credenciales del usuario **admin** se accedió al panel principal de FreeIPA, cuyo aspecto se muestra en la **Figura 23**.

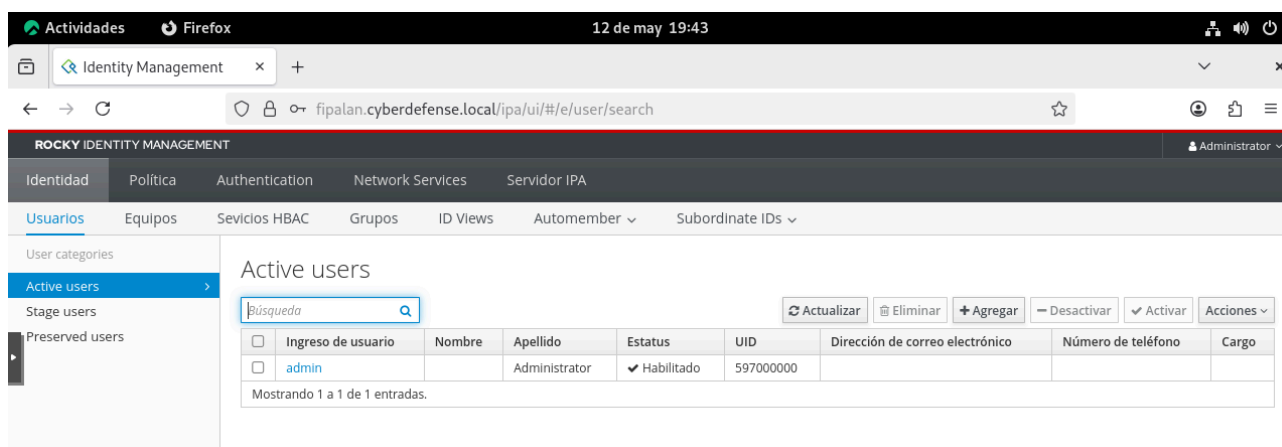


Figura 23: Panel principal - FreeIPA

Con el servidor de identidades operativo, el siguiente paso fue desplegar el sistema de monitorización de la red interna. Para ello, se instaló Wazuh en una máquina virtual independiente, que se encarga de centralizar los logs y generar alertas sobre el servidor y las estaciones de trabajo una vez han sido integrados en el dominio.

4.2. SIEM LAN

Para la monitorización de los clientes simulados, se desplegó un servidor Wazuh en la red LAN siguiendo el mismo procedimiento de instalación que en la DMZ, con las especificaciones de la **Figura 24**.

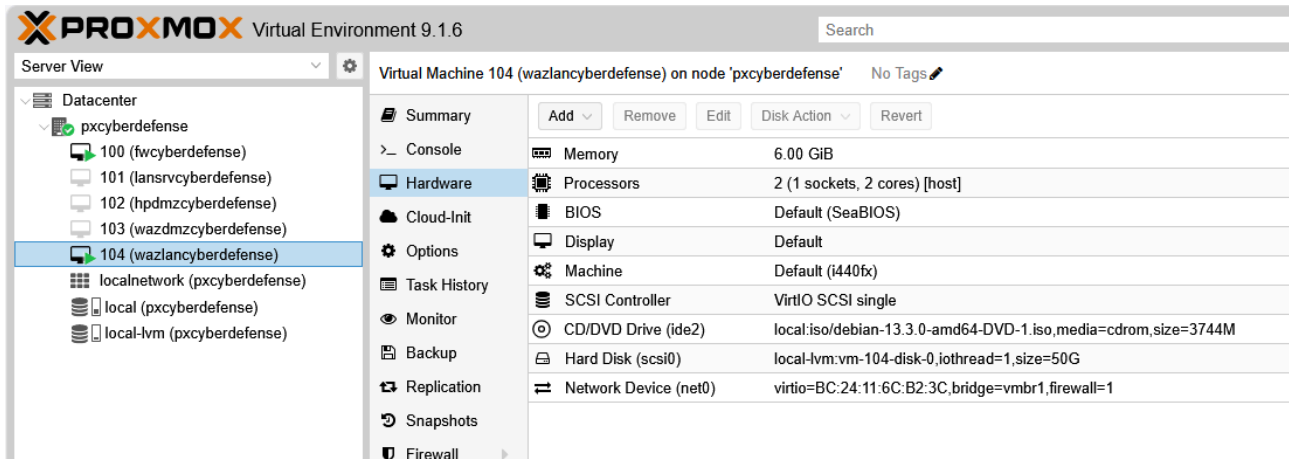


Figura 24: Máquina virtual - SIEM LAN

Se aplicó el mismo procedimiento de instalación y actualización de Debian 13 que en el honeypot. Una vez instalado el sistema, se configuró la red por DHCP, y se reservó en OPNsense una dirección IP estática para el servidor (192.168.0.2), tal como se muestra en la **Figura 25**.

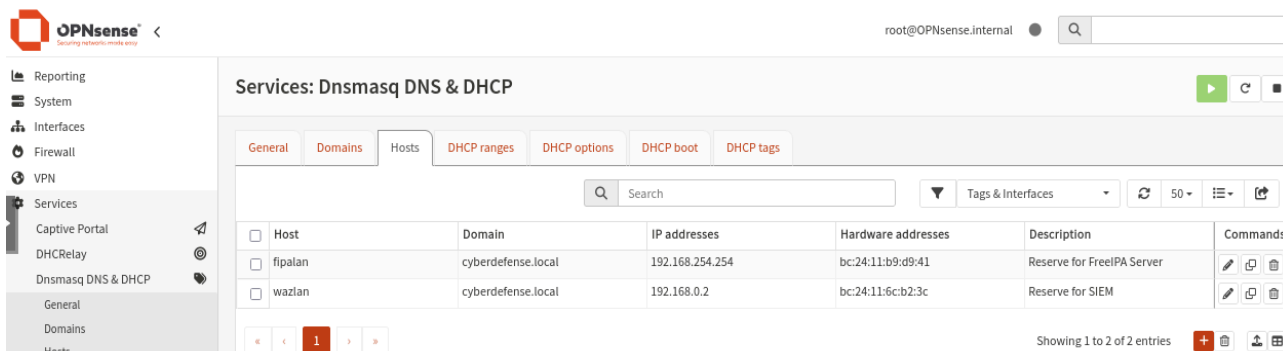


Figura 25: Reserva DHCP - SIEM LAN

A continuación, se configuraron los repositorios oficiales de Debian y se actualizó el sistema siguiendo la documentación oficial de la página web de Debian.

Con la máquina preparada, se instaló Wazuh en modo todo en uno (manager, indexer y dashboard) siguiendo el script oficial de instalación. Los tres componentes y sus funciones ya han sido descritos en el [apartado 3.2](#).

Una vez finalizada la instalación, se accedió al dashboard de Wazuh a través de la URL **https://192.168.0.2**. En la **Figura 26** se muestra la pantalla tras haber iniciado sesión.

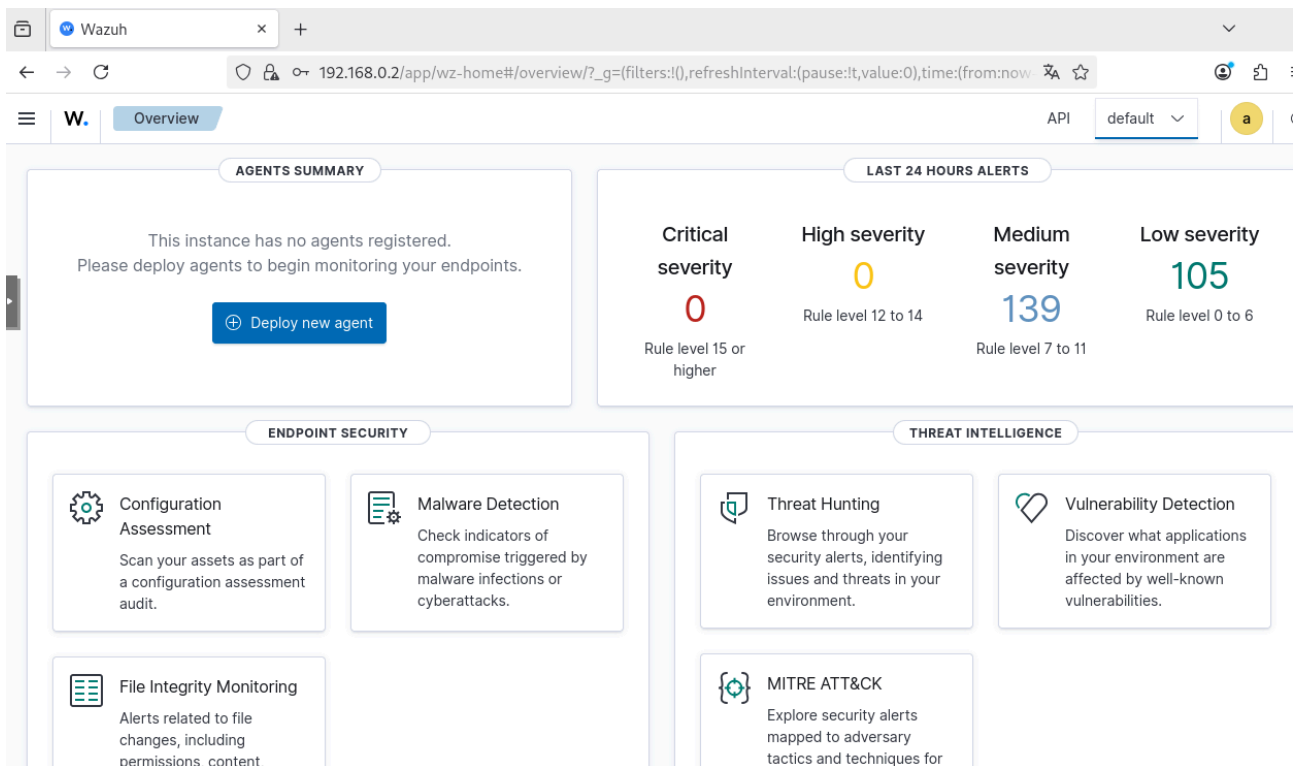


Figura 26: Wazuh Dashboard - SIEM LAN

Tras la instalación de la estación de trabajo, que se puede ver en el [siguiente punto](#), se desplegaron los agentes de wazuh tanto en la estación de trabajo como en el servidor FreeIPA, se enrolaron mediante el comando generado desde la parte de **Deploy new agent** en el dashboard de wazuh, al igual que para la DMZ.

En la **Figura 27** se muestra como quedan los agentes después de unir los dispositivos de la LAN.

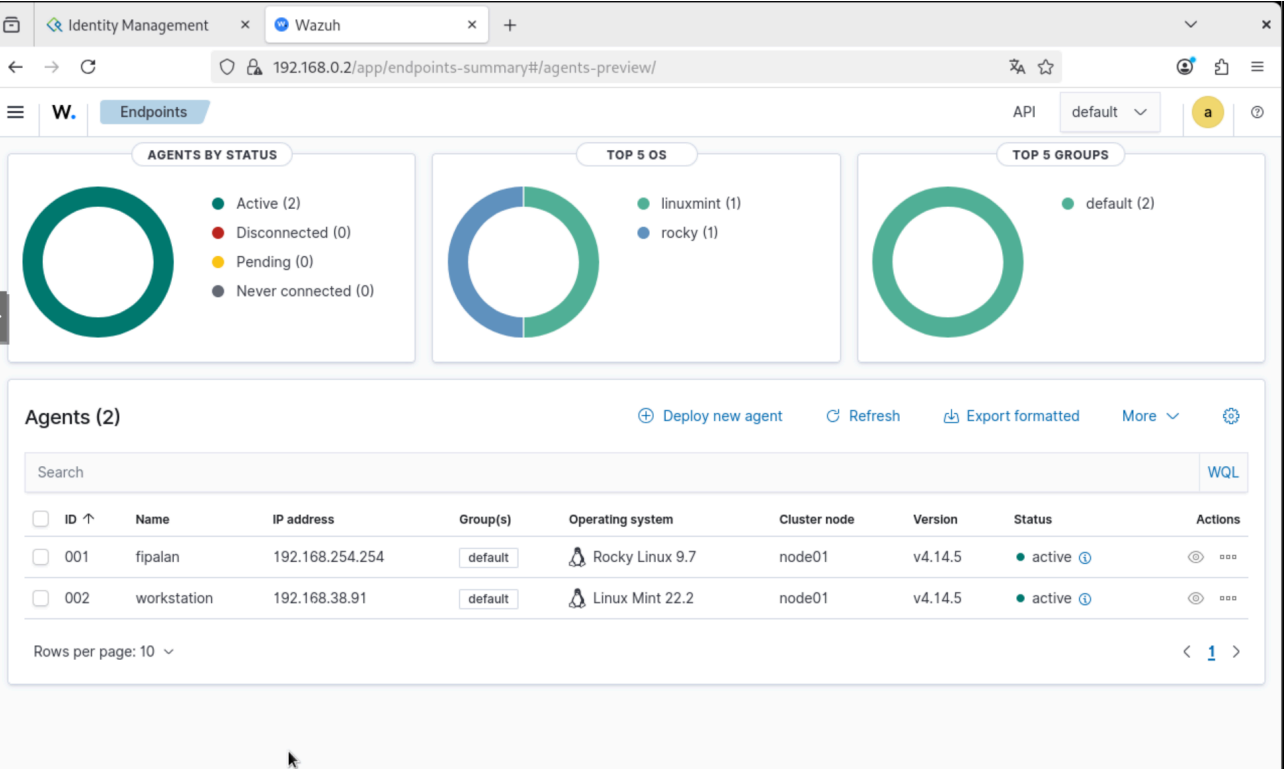


Figura 27: Agentes - SIEM LAN

4.3. Estación de trabajo

A continuación, se procedió a preparar una máquina para la simulación de lo que sería un equipo de un puesto de trabajo de la empresa.

Para ello, se creó una máquina virtual personalizada en la que se pueden ver sus especificaciones en la **Figura 28**.

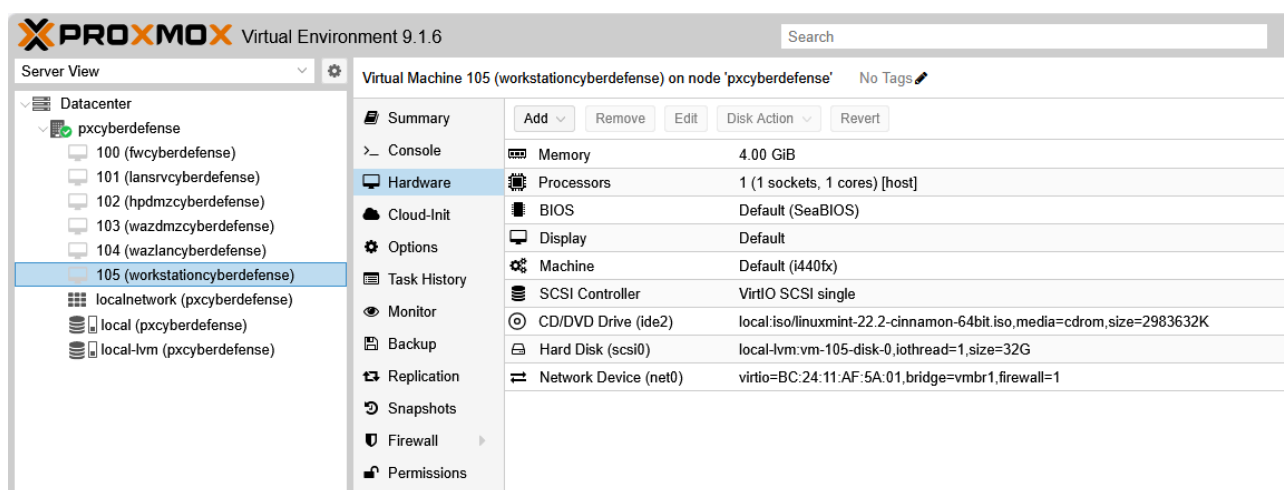


Figura 28: Máquina virtual - Est. Trabajo

Para los puestos de trabajo se optó por la distribución Linux Mint, con una interfaz amigable parecida a Windows pero con todas las ventajas y libre uso de Linux.

Para la instalación de Linux Mint se siguió el [manual de instalación](#) ubicado en la página web oficial.

Tras la instalación se configuró automáticamente la interfaz de red con una dirección automática dada por el servidor DHCP de OPNsense, tal y como se muestra en la **Figura 29**.

```
cyberad@workstationcybdef:~  
cyberad@workstationcybdef:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether bc:24:11:af:5a:01 brd ff:ff:ff:ff:ff:ff  
    altname enp0s18  
    inet 192.168.38.91/16 brd 192.168.255.255 scope global dynamic noprefixroute ens18  
        valid_lft 85640sec preferred_lft 85640sec  
    inet6 fe80::3eab:d11a:5796:6796/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
cyberad@workstationcybdef:~$
```

Figura 29: Interfaz de red - Workstation

Una vez configurada la red, se actualizaron los repositorios y los paquetes del sistema siguiendo la documentación oficial de Linux Mint.

A continuación, se procedió a integrar la máquina en el dominio centralizado de Cyber Defense S.L. Para ello, se instaló el paquete **freeipa-client** y se editaron los archivos de configuración necesarios.

El paquete se instaló con el comando: **\$ sudo apt install freeipa-client -y.**

En primer lugar, se definió el nombre completo de la máquina (FQDN) modificando los archivos del sistema **/etc/hostname** y **/etc/hosts**, estableciendo el FQDN en **workstationcybdef.cyberdefense.local**, para el equipo que simula una estación de trabajo.

Además, se añadió la siguiente entrada en el archivo **/etc/hosts** para garantizar la resolución del servidor FreeIPA:

```
$ echo "192.168.254.254 fipalan.cyberdefense.local fipalan" | sudo tee -a /etc/hosts
```

A continuación, se ejecutó el asistente de configuración del cliente de FreeIPA con los siguientes parámetros:

```
$ sudo ipa-client-install --hostname=$(hostname -f)
--domain cyberdefense.local \
--server fipalan.cyberdefense.local \
--realm CYBERDEFENSE.LOCAL \
--mkhomedir
```

Durante el proceso se solicitó las credenciales de un usuario administrador con permisos para integrar clientes en el dominio, completando la integración sin errores. En la **Figura 30** se muestra el resultado de la instalación, donde puede observarse el mensaje **Client configuration complete**.


```

cyberad@workstationcybdef: ~
Autodiscovery of servers for failover cannot work with this configuration.
If you proceed with the installation, services will be configured to always access the discovered
all operations and will not fail over to other servers in case of failure.
Proceed with fixed values and no DNS discovery? [no]: yes
Do you want to configure chrony with NTP server or pool address? [no]:
Client hostname: workstationcybdef.cyberdefense.local
Realm: CYBERDEFENSE.LOCAL
DNS Domain: cyberdefense.local
IPA Server: fipalan.cyberdefense.local
BaseDN: dc=cyberdefense,dc=local

Continue to configure the system with these values? [no]: yes
Synchronizing time
No SRV records of NTP servers found and no NTP server or pool address was provided.
Using default chrony configuration.
Attempting to sync time with chronyc.
Time synchronization was successful.
User authorized to enroll computers: admin
Password for admin@CYBERDEFENSE.LOCAL:
Successfully retrieved CA cert
  Subject: CN=Certificate Authority,O=CYBERDEFENSE.LOCAL
  Issuer: CN=Certificate Authority,O=CYBERDEFENSE.LOCAL
  Valid From: 2026-04-27 09:52:06+00:00
  Valid Until: 2046-04-27 09:52:06+00:00

Enrolled in IPA realm CYBERDEFENSE.LOCAL
Created /etc/ipa/default.conf
Configured /etc/sss/sss.conf
Systemwide CA database updated.
Could not update DNS SSHFP records.
SSSD enabled
/etc/ldap/ldap.conf does not exist.
Failed to configure /etc/openldap/ldap.conf
Unable to find 'admin' user with 'getent passwd admin@cyberdefense.local!'
Unable to reliably detect configuration. Check NSS setup manually.
Configured /etc/ssh/ssh_config
/etc/ssh/sshd_config not found, skipping configuration
Configuring cyberdefense.local as NIS domain.
Configured /etc/krb5.conf for IPA realm CYBERDEFENSE.LOCAL
Client configuration complete.
The ipa-client-install command was successful

```

Figura 30: Integración de la estación de trabajo en el dominio - FreeIPA

Una vez integrada la máquina en el dominio, se configuró el gestor de inicio de sesión **LightDM** para permitir el acceso con usuarios del dominio. Para ello, se editó el archivo `/etc/lightdm/lightdm.conf`, añadiendo las siguientes líneas:

```

[Seat:*]
greeter-hide-users=true
greeter-show-manual-login=true

```

Esta configuración oculta la lista de usuarios locales y muestra un campo de texto para introducir manualmente el nombre de usuario, permitiendo iniciar sesión con cualquier usuario del dominio.

5. Conexión exterior

5.1. EC2 - Amazon Web Services

5.1.1. Configuración inicial de la instancia

Para permitir el acceso desde internet al honeypot, se desplegó una instancia **EC2** en la nube de **Amazon Web Services (AWS)**. Para ello, se registró una cuenta en la plataforma utilizando el nivel **Free Tier**, que proporciona crédito inicial para servicios durante los primeros doce meses. Se optó por una instancia de tipo **t3.large**, cuyas características principales son **2 vCPUs** y **8 GB de RAM**, recursos más que suficientes para alojar el túnel WireGuard y el redireccionamiento de tráfico.

Además, se reservó y asoció una **dirección IP elástica (Elastic IP)** a la instancia, lo que garantiza una dirección pública fija para acceder siempre al mismo punto desde internet. Tanto la instancia como la IP elástica se desplegaron en la región **Europe (Spain) / eu-south-2**, con el objetivo de minimizar la latencia y maximizar la velocidad de las comunicaciones entre la EC2 y la infraestructura local de Cyber Defense S.L.

Una vez creada la instancia, con sistema operativo Debian 13, se procedió a actualizar los repositorios y paquetes del sistema siguiendo la guía oficial de actualización.

Tras las actualizaciones correspondientes, se cambió el puerto de administración SSH al 20208 para reducir el impacto de los ataques automatizados y proteger el acceso remoto real, además de restringir el acceso SSH únicamente a la IP del equipo de administración en el grupo de seguridad de la EC2.

Un **grupo de seguridad** en **AWS** actúa como un **firewall virtual a nivel de instancia** que controla el tráfico entrante y saliente mediante reglas basadas en protocolo, puerto y

origen/destino. A diferencia de las ACL de red, los grupos de seguridad son **stateful**, lo que significa que si se permite el tráfico entrante desde una IP, el tráfico de respuesta saliente se permite automáticamente sin necesidad de reglas explícitas.

En la **Figura 31** se puede consultar cómo quedó el grupo de seguridad de la instancia tras la configuración para permitir las conexiones necesarias.

Reglas de entrada (14)

Q

Buscar

Administrar etiquetas

Editar reglas de entrada

<

1

>

☐	Name	ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen	Descripción
☐	-	sgr-08a4307a47a12c0cc	IPv4	TCP personalizado	TCP	8873	0.0.0.0/0	RSYNC for Honeypot C...
☐	-	sgr-099824189109a0012	IPv4	TCP personalizado	TCP	2323	0.0.0.0/0	TELNET ALT for Honey...
☐	-	sgr-05fedda7bbc1c96af	IPv4	LDAP	TCP	389	0.0.0.0/0	LDAP for Honeypot Co...
☐	-	sgr-027f0984ae39c6343	IPv4	UDP personalizado	UDP	65534	0.0.0.0/0	WireGuard Connection
☐	-	sgr-04638a32346b23b7f	IPv4	TCP personalizado	TCP	20208	79.116.123.56/32	SSH ALT for Secure RA
☐	-	sgr-00432f068af404975	IPv4	Todos los ICMP IPv4	ICMP	Todo	0.0.0.0/0	ICMP Traffic for Tracero...
☐	-	sgr-0730bb03d52dd4535	IPv4	TCP personalizado	TCP	5901	0.0.0.0/0	VNC for Honeypot Con...
☐	-	sgr-07aeb6cce79091a1	IPv4	TCP personalizado	TCP	8443	0.0.0.0/0	HTTPS for Honeypot C...
☐	-	sgr-0370c2b411d27d298	IPv4	TCP personalizado	TCP	8080	0.0.0.0/0	HTTP for Honeypot Co...
☐	-	sgr-09302ea06988885d1	IPv4	MSSQL	TCP	1433	0.0.0.0/0	MSSQL for Honeypot C...
☐	-	sgr-0af08ad3d4f81d12	IPv4	HTTPS	TCP	443	0.0.0.0/0	Main Web Page
☐	-	sgr-0c7f951e70de3fa91	IPv4	TCP personalizado	TCP	2222	0.0.0.0/0	SSH for Honeypot Con...
☐	-	sgr-00a9b93b84c668a2c	IPv4	RDP	TCP	3389	0.0.0.0/0	RDP for Honeypot Con...
☐	-	sgr-08a977c491e68dc99	IPv4	TCP personalizado	TCP	636	0.0.0.0/0	LDAPS for Honeypot C...

Figura 31: Grupo de seguridad - EC2

El primer paso para la conexión con la infraestructura fue habilitar la capacidad de la EC2 para actuar como un router, es decir, recibir el tráfico de Internet y enviarlo a través del túnel WireGuard al honeypot.

Para activar el reenvío de paquetes de forma permanente se creó un fichero de configuración llamado **99-forward.conf** ubicado en **/etc/sysctl.d**, con el siguiente contenido:

```
net.ipv4.ip_forward=1
```

Por último, se aplicó el cambio con el comando **\$ sudo sysctl --system**.

Con el reenvío de paquetes activado, el siguiente paso fue instalar y configurar WireGuard para la creación del túnel seguro, esto se hizo con el comando `$ sudo apt install wireguard -y`.

Tras la instalación se generaron el par de claves pública-privada, que identifican a la EC2 en el túnel. Esto se consiguió con el comando `$ wg genkey | tee privatekey | wg pubkey > publickey`.

Con esto las claves se guardaron en cada archivo correspondiente, tras esto se creó el fichero de configuración de WireGuard `/etc/wireguard/wg0.conf` cuyo contenido se puede consultar en el [Anexo E](#).

Se definió en este archivo la dirección de la EC2 dentro del túnel (**10.10.0.1**) y la clave privada recién creada, para los peers o conexiones se permitieron las subredes **10.10.0.2/32** (ip de OPNsense en el túnel) y **172.26.10.0/24** (red de la DMZ). Se establecieron rutas basadas en reglas de iptables que redirigen las conexiones recibidas desde la EC2 al honeypot, reenviando todos los paquetes, pero excluyendo únicamente el puerto 20208 destinado a la administración remota. Además, se configuró la clave pública de OPNsense, que es quien recibe la conexión en la infraestructura local, y se añadió una clave pre compartida como medida de seguridad adicional. Por último se cambió el puerto por defecto de escucha de WireGuard al 65534 y se estableció el **PersistentKeepalive** en 15 segundos para mantener activo el túnel.

Para iniciar y habilitar el túnel al encendido del equipo se hizo con el comando `$ sudo systemctl enable wg-quick@wg0 && sudo systemctl start wg-quick@wg0`.

Para verificar el estado del túnel se puede usar `sudo systemctl status wg-quick@wg0` o `sudo wg show`, tal y como se muestra en la **Figura 32**.

```
cyberad@ip-172-31-28-132: ~  
cyberad@ip-172-31-28-132:~$ sudo wg show  
interface: wg0  
  public key: KDJgp6s4du9QZekEow3tSRjg7KeRNgZwex2NnYIbSUA=  
  private key: (hidden)  
  listening port: 65534  
  
peer: lyvho3qr2RScE54mddZ03QiNWzdhDm6E1bgJRciycQs=  
  preshared key: (hidden)  
  endpoint: 79.116.242.42:51820  
  allowed ips: 10.10.0.2/32, 172.26.10.0/24  
  latest handshake: 1 minute, 1 second ago  
  transfer: 874.03 KiB received, 1.12 MiB sent  
  persistent keepalive: every 15 seconds  
cyberad@ip-172-31-28-132:~$ sudo systemctl status wg-quick@wg0 --no-pager  
● wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0  
   Loaded: loaded (/usr/lib/systemd/system/wg-quick@.service; enabled; preset: enabled)  
   Active: active (exited) since Sat 2026-05-23 12:39:55 UTC; 44min ago  
  Invocation: fd7a5778b69a4bbeae817da69c095e72  
     Docs: man:wg-quick(8)  
           man:wg(8)  
           https://www.wireguard.com/  
           https://www.wireguard.com/quickstart/  
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8  
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8  
   Process: 692 ExecStart=/usr/bin/wg-quick up wg0 (code=exited, status=0/SUCCESS)  
  Main PID: 692 (code=exited, status=0/SUCCESS)  
  Mem peak: 2.8M  
    CPU: 93ms  
  
May 23 12:39:54 ip-172-31-28-132 wg-quick[692]: [#] ip link add wg0 type wireguard
```

Figura 32: Estado túnel WireGuard - EC2

5.1.2. Configuración de OPNsense para el túnel WireGuard

Una vez configurada la EC2, se procedió a habilitar el otro extremo del túnel en el firewall local OPNsense.

El servicio de WireGuard en esta versión ya estaba activo por defecto pero en el caso de que no lo estuviese, es necesario descargar el plugin desde **System** → **Firmware** → **Plugins** e instalar **os-wireguard**.

A continuación, en la pestaña de **VPN** → **WireGuard** → **Instances** se habilitó WireGuard y se creó una nueva instancia con los siguientes parámetros; se le dio el nombre de **AWS-EC2** a la instancia, se configuró la clave pública y privada de OPNsense y se

estableció la dirección **10.10.0.2** para la interfaz del túnel, en la **Figura 33** se podrá ver creada.

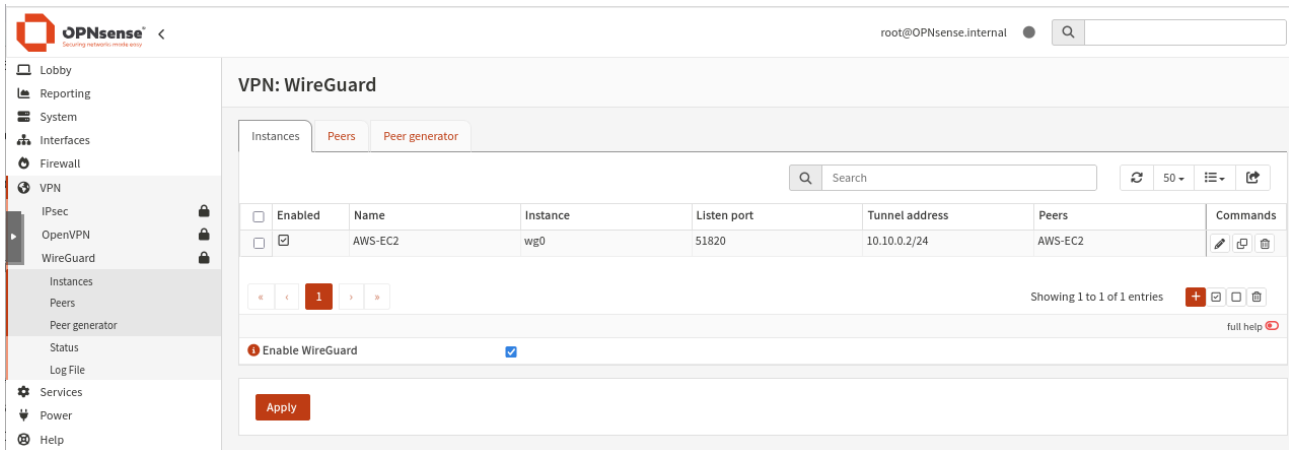


Figura 33: Instancia WireGuard - OPNsense

En la configuración de la conexión o el peer, en **VPN → WireGuard → Peers**, se añadió una entrada con los siguientes datos; se le dio mismo nombre que en la instancia, se configuró la clave pública de la EC2 y la clave pre compartida, se permitió la IP **10.10.0.1/32** (ip de EC2 en el túnel), y se configuró la IP pública de la instancia EC2, además del puerto configurado anteriormente como **65534**, por último se estableció el **PersistentKeepalive** en 15 segundos, en la **Figura 34** se podrá ver creada.

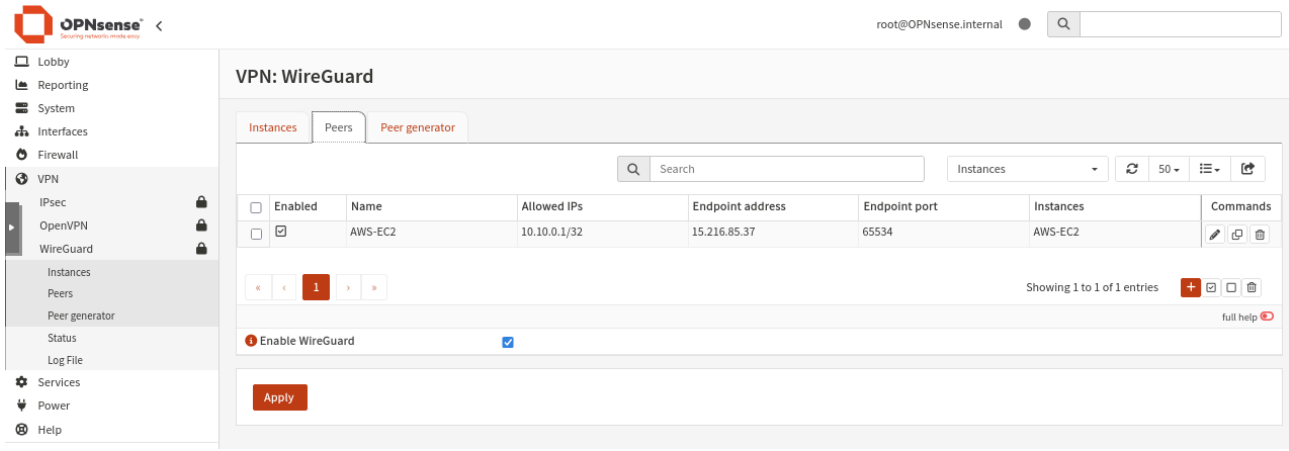


Figura 34: WireGuard Peer - OPNsense

Por último se creó una regla en la nueva interfaz de WireGuard para permitir el paso del tráfico únicamente a la red de la zona desmilitarizada, esto se hizo en **Firewall** → **Rules** → **WG**, en la **Figura 35** se puede ver la regla creada.

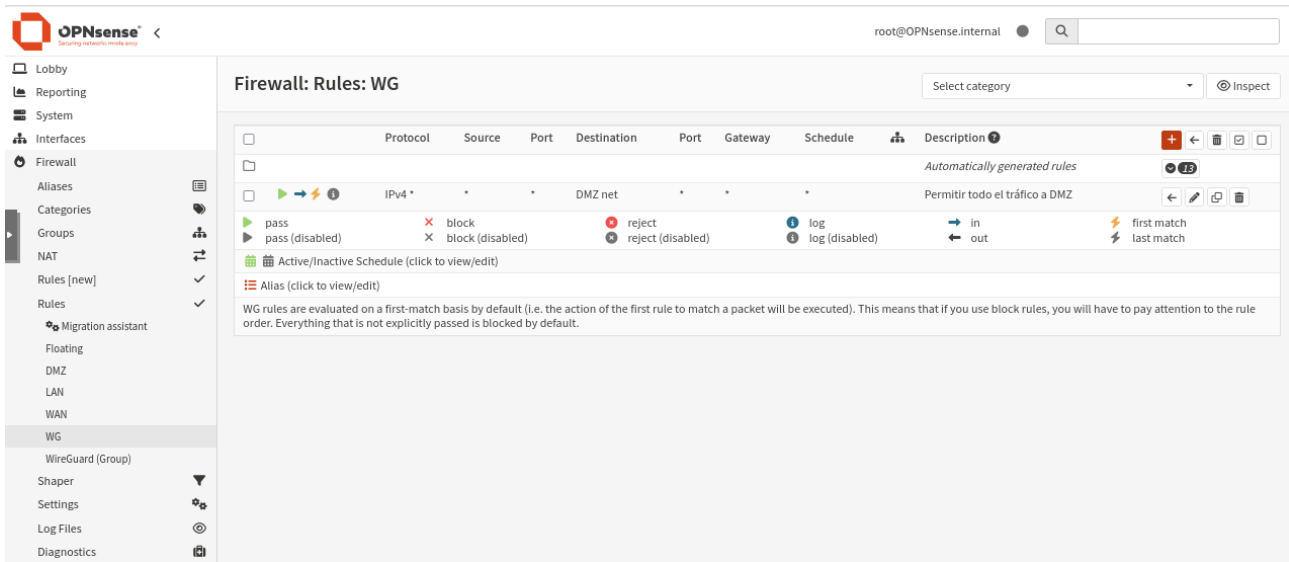


Figura 35: Regla conexión red WG con DMZ - OPNsense

Para verificar la conexión del túnel en la parte de la infraestructura local, se accede a **VPN** → **WireGuard** → **Status**, y debe aparecer operativo como en la **Figura 36**.

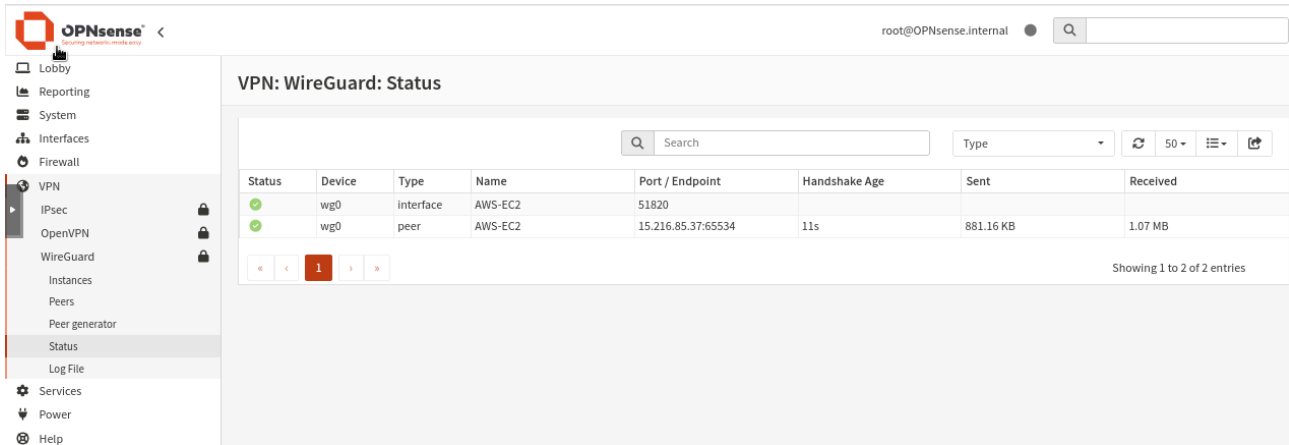


Figura 36: Estado túnel WireGuard - OPNsense

5.1.3. Configuración de la monitorización en EC2

Con el objetivo de registrar las IPs públicas de los atacantes, se decidió instalar el agente de Wazuh en la instancia EC2 y configurarlo para que reportara sus logs al servidor Wazuh de la DMZ. De esta forma, los eventos generados viajarán a través del propio túnel WireGuard hasta el SIEM.

El primer paso fue configurar una regla de iptables basada en la tabla **raw**, que permite capturar los paquetes de red dirigidos al honeypot antes de que sean enrutados. De esta manera, los paquetes quedan registrados en un archivo de log para su posterior análisis por parte de Wazuh.

El comando de **iptables** utilizado para conseguir esto fue el siguiente: **\$ sudo iptables -t raw -A PREROUTING -i ens5 -p tcp -m multiport --dports 3306,5353,8443,2222,8873,636,2121,389,1433,8080,3389,5901,2323 -j LOG --log-prefix "IPTables-Attack-Log: " --log-level 4.**

Tras verificar que se reciben los logs en **/var/log/kern.log**, se desplegó el agente de wazuh con el comando que se genera automáticamente desde el dashboard y en el archivo de configuración del agente de wazuh, ubicado en **/var/ossec/etc/ossec.conf** se añadió un bloque localfile para la recolección de logs de este archivo, tal como se detalla en el [Anexo E](#), en la **Figura 37** se muestra el resultado final de los agentes del SIEM DMZ.

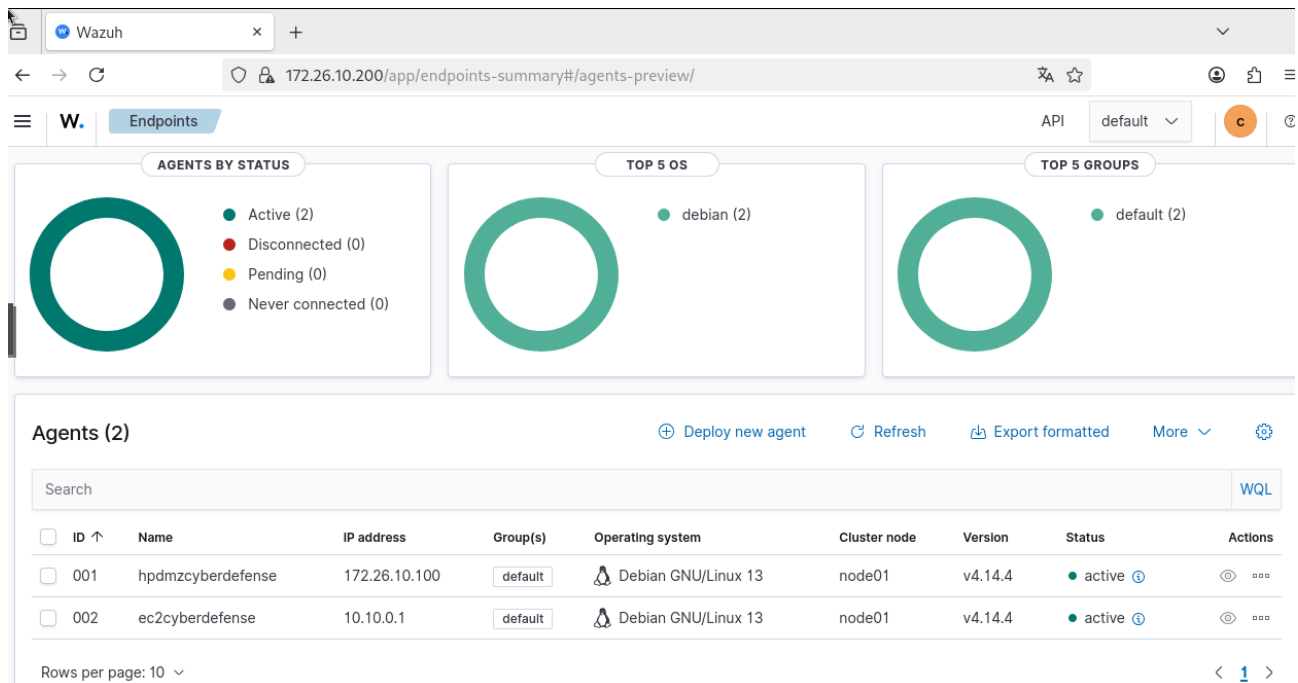


Figura 37: Agente registrado en SIEM DMZ - EC2

Una vez los logs comenzaron a llegar al SIEM, se definieron reglas personalizadas para generar alertas ante la actividad detectada. Para ello se editó el archivo de reglas locales `/var/ossec/etc/rules/local_rules.xml` y se añadieron las reglas correspondientes, que pueden consultarse en el [Anexo E](#), junto con el resto de reglas destinadas al honeypot. En la **Figura 38** se muestran las reglas operativas y funcionando con conexiones reales.

	W.	Threat Hunting	API	default	c
	May 23, 2026 @ 14:48:20.4...	ec2cyberdefense	Repeated external probing detected from 20.64.104.132	12	140030
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	Repeated external probing detected from 20.64.104.132	12	140030
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	Repeated external probing detected from 20.64.104.132	12	140030
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015
	May 23, 2026 @ 14:48:26.4...	ec2cyberdefense	BASE: External connection attempt detected on EC2 from 20.64.104.132	6	140010
	May 23, 2026 @ 14:48:25.5...	hpdmczyberdefense	HTTP request to honeypot web service	7	100030
	May 23, 2026 @ 14:46:58.4...	ec2cyberdefense	External connection attempt detected on EC2 from 20.64.104.132	8	140015

Figura 38: Alertas SIEM - EC2

Regla 140010 (nivel 6) – Actúa como regla base que se activa con cada evento IPTables-Attack-Log. Sirve como disparador para las reglas de frecuencia posteriores, evitando una avalancha de alertas por cada paquete individual.

Regla 140015 (nivel 8) – Se activa cuando la misma IP origen dispara la regla base 2 veces en 2 segundos. Detecta escaneos rápidos o intentos de conexión repetitivos.

Regla 140030 (nivel 12) – Se activa cuando la regla 140015 se dispara 5 veces en 30 segundos desde la misma IP. Indica un escaneo persistente o una posible fuerza bruta en curso.

d) Conclusiones

1. Cumplimiento de los objetivos

El proyecto alcanzó la totalidad de los objetivos planteados inicialmente. Se diseñó e implementó una infraestructura de red segmentada en tres zonas (WAN, DMZ y LAN), con políticas de firewall en OPNsense que aíslan completamente la DMZ de la red interna, impidiendo cualquier intento de pivoteo por parte de un atacante que pudiera comprometer el honeypot. El honeypot Trapster se desplegó en la DMZ simulando una decena de servicios reales (SSH, FTP, RDP, HTTP, HTTPS, Telnet, LDAP, MySQL, MSSQL, VNC, DNS y Rsync) y registró con éxito múltiples intentos de conexión procedentes de Internet a través del túnel WireGuard establecido con una instancia EC2 de AWS.

Los dos sistemas Wazuh (uno en DMZ y otro en LAN) centralizan y analizan los logs del honeypot, de la EC2 y de los equipos internos, generando alertas clasificadas según el marco MITRE ATT&CK, lo que facilita la labor de los analistas de seguridad. Los dashboards configurados, incluyendo el panel personalizado "Attack Monitor", muestran en tiempo real la actividad maliciosa capturada, incluyendo ranking de IPs atacantes, procedencia geográfica y evolución temporal de los ataques. Las reglas de firewall de OPNsense garantizan el aislamiento total entre la DMZ y la LAN, y el túnel WireGuard ha permanecido estable durante todo el período de exposición. Finalmente, los datos recopilados durante las semanas de pruebas han permitido extraer conclusiones sobre las tendencias actuales de ciberataques, cumpliendo así el objetivo de analizar la información capturada.

2. Resultados más relevantes

Durante el período de exposición del honeypot se registraron más de un centenar de intentos de conexión desde múltiples IP's únicas. El análisis de los logs reveló que el 60% de los ataques se dirigieron a los puertos 2222 (SSH) y 8080 (HTTP), lo que confirma que estos servicios siguen siendo los más buscados por los atacantes automatizados. La mayoría de las IPs atacantes procedían de proveedores de hosting y centros de datos (DigitalOcean, OVH, Storm Industries), con puntuaciones de confianza de abuso elevadas en AbusIPDB, lo que indica que los atacantes utilizan infraestructura en la nube de bajo costo para lanzar sus escaneos.

La implementación de reglas de frecuencia en Wazuh (140010, 140015, 140030) ha demostrado ser eficaz para detectar escaneos activos y posibles fuerzas brutas, elevando el nivel de alerta de 6 a 12 en función de la insistencia del atacante. La integración de la estación de trabajo Linux Mint en el dominio FreeIPA ha permitido centralizar la autenticación y las políticas de acceso, mientras que el agente de Wazuh en la LAN monitoriza la actividad de los equipos internos sin interferir con la DMZ. El túnel WireGuard ha funcionado de manera estable durante todo el proyecto, con un handshake continuo y latencias mínimas, gracias a la configuración de PersistentKeepalive y al cambio de puerto a 65534 para evitar escaneos genéricos.

3. Dificultades encontradas

La principal dificultad técnica fue lograr que los logs del contenedor Trapster persistieran y tuvieran un formato compatible con Wazuh. El contenedor, por defecto, escribía los logs en su sistema interno sin mantenerlos tras su reinicio, y el formato de timestamp no era el estándar ISO 8601 que Wazuh espera para interpretar correctamente las fechas. Se solucionó modificando el archivo docker-compose.yml para montar un volumen que mapea la carpeta de logs del repositorio con la ruta interna del contenedor, y editando el archivo logger.py de Trapster para generar las marcas de tiempo en el formato requerido.

Otra dificultad relevante fue el conflicto de puertos 443 entre FreeIPA y Wazuh LAN, ya que ambos servicios requieren este puerto para sus respectivas interfaces web. Se barajó la posibilidad de alojarlos en la misma máquina, pero se descartó por motivos de seguridad y operación. La solución fue desplegar FreeIPA y Wazuh LAN en máquinas virtuales independientes, una decisión que además refuerza la segmentación de servicios y reduce la superficie de ataque.

Por último, el ajuste fino de las reglas de iptables en la EC2 requirió varias pruebas para capturar únicamente los paquetes dirigidos al honeypot sin saturar los logs con tráfico irrelevante. La lista de puertos a monitorizar se fue refinando progresivamente hasta alcanzar el equilibrio deseado.

4. Lecciones aprendidas

La principal lección aprendida es que, con herramientas de código abierto y una planificación adecuada, es posible construir una infraestructura de ciberseguridad profesional con un presupuesto mínimo. La combinación de Proxmox, OPNsense, Trapster, Wazuh y WireGuard ofrece una solución completa que no solo detecta ataques, sino que también proporciona información valiosa para anticiparse a futuras intrusiones.

La documentación detallada de cada paso ha sido clave para resolver los problemas rápidamente y para que el sistema sea mantenible en el futuro. También se ha aprendido la importancia de la segmentación de red y del aislamiento de servicios críticos como FreeIPA.

Otro aprendizaje importante es que los atacantes automatizados no distinguen entre un servidor real y un honeypot bien configurado; simplemente escanean la totalidad del espacio de direcciones IP en busca de servicios vulnerables. Por lo tanto, un honeypot correctamente desplegado es una herramienta de inteligencia de amenazas muy efectiva, independientemente del tamaño de la organización.

5. Reflexión final

Este proyecto demuestra que una PYME o un autónomo pueden disponer de un sistema de inteligencia de amenazas robusto y económico, sin necesidad de costosas licencias comerciales. El carácter modular de la arquitectura permite escalar fácilmente añadiendo más clientes en la LAN o más señuelos en la DMZ. Personalmente, el desarrollo de este trabajo ha reforzado mi convicción de que la seguridad proactiva, basada en monitorización continua, es una de las estrategias más efectivas para proteger activos digitales en la actualidad. La experiencia adquirida en la resolución de los problemas técnicos, especialmente los relacionados con la persistencia de logs en contenedores y la segmentación de servicios, supone un valor añadido para futuros despliegues profesionales.

e) Nuevas propuestas

Línea 1: Integración de identidad híbrida y control de accesos unificado

Extender FreeIPA para que actúe como núcleo de identidad central, estableciendo confianza con Active Directory on-premise y sincronización con Azure AD/Entra ID. Sobre esto, construir una VPN corporativa completa con autenticación basada en esos dominios, registro de todas las conexiones, políticas de acceso por grupo y revocación instantánea. Añadir gestión de sesiones privilegiadas (PAM) y autenticación multifactor para todos los accesos administrativos a la infraestructura del SOC.

Línea 2: Centro de operaciones completo (SOC + inteligencia + respuesta)

Integrar el Wazuh actual con una plataforma MISP propia, alimentada con feeds de inteligencia abiertos y cerrados, de forma que las alertas del honeypot y de los sensores internos se enriquezcan automáticamente con contexto de amenazas conocidas. Incorporar un EDR en todos los endpoints (tanto en LAN como en DMZ) que permita acciones de respuesta en tiempo real. Añadir un orquestador SOAR ligero para automatizar playbooks como bloqueos de IP, aislamiento de equipos o generación de tickets. Todo ello coordinado en un único cuadro de mando ejecutivo con informes automáticos para dirección y auditorías.

Línea 3: Gobierno del riesgo y continuidad del negocio

Desplegar un sistema de gestión de riesgos y cumplimiento normativo (ISO 27001, ENS, GDPR) que se alimente directamente de los hallazgos de Wazuh, vulnerabilidades detectadas y eventos de seguridad. Establecer una política de backups completos y probados periódicamente para todas las VM, configuraciones y logs, con almacenamiento fuera de la red principal y recuperación ante desastres documentada. Incluir simulaciones de adversarios (breach and attack simulation) para validar la efectividad de las defensas y los procedimientos de respuesta.

Línea 4: Extensión multicloud y sensores remotos

Extender la infraestructura del SOC a múltiples sedes físicas y entornos cloud (AWS, Azure, etc.) mediante sensores ligeros que envíen telemetría al Wazuh central a través de la red privada virtual. Esto incluye desplegar honeypots adicionales en diferentes regiones geográficas y proveedores, así como agentes de monitorización en cargas de trabajo de producción. Consolidar todos los logs, alertas y métricas en una única vista federada, permitiendo la detección de ataques distribuidos y movimientos laterales entre entornos.

Línea 5: Sandbox interactivo y análisis de malware

Desplegar un entorno sandbox aislado (Cuckoo o CAPE) integrado con el SIEM, de forma que cualquier artefacto descargado por el honeypot o alerta sospechosa sea ejecutado automáticamente en un entorno controlado para observar su comportamiento real. Complementar con Cowrie (honeypot SSH mejorado) que simule un sistema comprometido, registrando todas las interacciones del atacante una vez dentro: comandos ejecutados, descargas, movimientos laterales y herramientas utilizadas. Ambos sistemas se alimentarían de las alertas de Wazuh y enriquecerían los incidentes con informes detallados de actividad post-explotación, permitiendo a los analistas de Cyber Defense comprender las tácticas reales del adversario y generar firmas personalizadas de detección.

5. Anexos

a) Referencias bibliográficas

1. Situación previa

Noticias e informes de seguridad →

<https://www.vozpopuli.com/espana/letalidad-ciberataque-pymes-cierre.html>

<https://emprendedores.es/gestion/pymes-ciberataque-2026/>

https://www.ospi.es/images/documentos/archivos/Google_Panorama-actual-de-la-ciberseguridad-en-Espana.pdf

https://www.hiscox.es/sites/spain/files/2025-10/HSX374_2025%20CRR%20Report_ES%20%281%29.pdf

2. Implementación

Proxmox

Guía oficial para establecer los repositorios de no suscrito en Proxmox:

https://pve.proxmox.com/wiki/Package_Repositories#sysadmin_no_subscription_repo

OPNsense

Guía oficial de actualización de OPNsense:

<https://docs.opnsense.org/manual/updates.html>

Debian 13

Guía oficial de actualización de Debian 13:

<https://www.debian.org/releases/stable/release-notes/upgrading.es.html#upgrading-packages>

Trapster

Sistema de logging en el honeypot:

<https://deepwiki.com/0xBallpoint/trapster-community/3.3-logging-infrastructure>

b) Glosario de términos

ACL (Access Control List) → Lista de reglas que define qué tráfico está permitido o denegado en un dispositivo de red. Se menciona en el contexto de los grupos de seguridad de AWS.

Bridge (puente de red) → Adaptador virtual que conecta dos o más segmentos de red. En Proxmox se usan vmbr0, vmbr1, etc., para comunicar máquinas virtuales entre sí o con la red física.

Date Histogram → Agregación utilizada en visualizaciones de datos para agrupar eventos por intervalos de tiempo (minuto, hora, día, etc.). Se emplea en el gráfico de barras del dashboard "Attack Monitor".

EC2 (Elastic Compute Cloud) → Servicio de computación en la nube de Amazon Web Services que permite alquilar máquinas virtuales. En el proyecto se usa una instancia t3.large como puerta de entrada pública.

EDR (Endpoint Detection and Response) → Solución de seguridad que monitoriza y responde a amenazas en dispositivos finales (endpoints), permitiendo acciones como aislamiento o bloqueo en tiempo real.

Free Tier → Nivel gratuito de AWS que ofrece crédito inicial y servicios limitados durante los primeros doce meses, utilizado para desplegar la instancia EC2 sin coste.

Handshake (WireGuard) → Intercambio de claves y establecimiento de conexión entre dos peers de WireGuard. Un handshake reciente indica que el túnel está activo.

iptables → Herramienta de administración de filtrado de paquetes en el núcleo Linux. Se usa en la EC2 para registrar (LOG) los paquetes dirigidos al honeypot.

ISO 8601 → Estándar internacional para la representación de fechas y horas (ej. 2026-05-18T18:12:25.012+0000). Wazuh requiere este formato para interpretar correctamente los timestamps.

KDC (Key Distribution Center) → Componente central de Kerberos que emite tickets de autenticación y gestiona las claves de sesión. Aparece en el estado de servicios de FreeIPA.

Kerberos – Protocolo de autenticación de red basado en tickets que permite la comunicación segura entre nodos sin transmitir contraseñas. FreeIPA lo utiliza como base de autenticación.

LightDM – Gestor de inicio de sesión utilizado en Linux Mint. Se configura para permitir el acceso manual con usuarios del dominio FreeIPA.

localfile – Configuración en los agentes de Wazuh que define qué archivos de log deben ser monitorizados, especificando la ruta y el formato (syslog, json, etc.).

MISP (Malware Information Sharing Platform) – Plataforma de código abierto para compartir inteligencia de amenazas (indicadores de compromiso, firmas, etc.). Se propone su integración futura.

MITRE ATT&CK – Marco de conocimiento global que categoriza las tácticas y técnicas utilizadas por ciberatacantes. Se usa para clasificar las alertas generadas por Wazuh.

PAM (Pluggable Authentication Modules) – Sistema de autenticación modular en Linux. Se menciona en la autenticación de Proxmox ("autenticación PAM estándar").

PAM (Privileged Access Management) – En el contexto de nuevas propuestas, se refiere a la gestión de accesos privilegiados, diferente del anterior. Controla sesiones administrativas con políticas elevadas.

PersistentKeepalive – Opción de WireGuard que envía paquetes keepalive periódicos (cada 15 segundos en el proyecto) para mantener el túnel activo a través de NAT y firewalls.

RAID (Redundant Array of Independent Disks) – Tecnología que combina múltiples discos para mejorar el rendimiento (RAID 0, 10) o la redundancia (RAID 1). Aparece en los requisitos de producción.

raw (tabla raw) – Tabla de iptables que procesa paquetes antes que cualquier otra, usada para registrar tráfico sin afectar el flujo normal. Se emplea para capturar paquetes dirigidos al honeypot.

Realm – Dominio de autenticación Kerberos que define un límite administrativo. En FreeIPA se configura como CYBERDEFENSE.LOCAL.

SOAR (Security Orchestration, Automation and Response) – Plataforma que automatiza la respuesta a incidentes mediante playbooks (bloqueo de IPs, aislamiento, tickets, etc.). Se propone como mejora futura.

Stateful – Propiedad de un firewall (o grupo de seguridad) que recuerda el estado de las conexiones activas, permitiendo automáticamente el tráfico de respuesta. Se menciona en AWS.

VPN (Virtual Private Network) – Red privada virtual que extiende una red local sobre una red pública (Internet) mediante un túnel cifrado. En el proyecto se usa WireGuard como VPN.

WireGuard – Protocolo de VPN moderno, ligero y de alto rendimiento, basado en criptografía de curvas elípticas. Se utiliza para el túnel entre la EC2 y OPNsense.

vmbr – Identificador de los bridges virtuales en Proxmox (vmbr0, vmbr1, vmbr2). Cada uno actúa como un conmutador virtual que conecta máquinas virtuales entre sí o con interfaces físicas.

c) Índices de tablas, figuras y diagramas

Tablas:

- ☐ [Tabla 1: Requisitos funcionales](#)
- ☐ [Tabla 2: Requisitos no funcionales](#)
- ☐ [Tabla 3: Requisitos del laboratorio](#)
- ☐ [Tabla 4: Requisitos estimados para producción](#)
- ☐ [Tabla 5: Requisitos software](#)
- ☐ [Tabla 6: Requisitos cloud](#)

Diagramas:

- ★ [Diagrama 1: Infografía del proyecto](#)
- ★ [Diagrama 2: Diagrama de Gantt del proyecto](#)
- ★ [Diagrama 3: Esquema de red de la infraestructura](#)

Figuras:

- [Figura 1: Adaptador de red - VMware](#)
- [Figura 2: Máquina virtual VMware - Proxmox](#)
- [Figura 3: Resumen de la instalación - Proxmox](#)
- [Figura 4: Adaptadores de red - Proxmox](#)
- [Figura 5: Lista de ISO's subidas - Proxmox](#)
- [Figura 6: Máquina virtual - OPNsense](#)
- [Figura 7: Interfaces en terminal - OPNsense](#)
- [Figura 8: Interfaces en la web - OPNsense](#)
- [Figura 9: Reglas DMZ - OPNsense](#)
- [Figura 10: Máquina virtual - Honeypot](#)
- [Figura 11: Interfaz de red - Honeypot](#)
- [Figura 12: Contenedor de Trapster - Honeypot](#)

- [Figura 13: Máquina virtual - SIEM DMZ](#)
- [Figura 14: Interfaz de red - SIEM DMZ](#)
- [Figura 15: Wazuh Dashboard - SIEM DMZ](#)
- [Figura 16: Agente registrado - SIEM DMZ](#)
- [Figura 17: Alertas Honeypot - SIEM DMZ](#)
- [Figura 18: Dashboard personalizado - SIEM DMZ](#)
- [Figura 19: Máquina virtual - FreeIPA](#)
- [Figura 20: Reserva DHCP - FreeIPA](#)
- [Figura 21: Pre-requisitos - FreeIPA](#)
- [Figura 22: Estado de los servicios de FreeIPA](#)
- [Figura 23: Panel principal - FreeIPA](#)
- [Figura 24: Máquina virtual - SIEM LAN](#)
- [Figura 25: Reserva DHCP - SIEM LAN](#)
- [Figura 26: Wazuh Dashboard - SIEM LAN](#)
- [Figura 27: Agentes - SIEM LAN](#)
- [Figura 28: Máquina virtual - Est. Trabajo](#)
- [Figura 29: Interfaz de red - Workstation](#)
- [Figura 30: Integración de la estación de trabajo en el dominio - FreeIPA](#)
- [Figura 31: Grupo de seguridad - EC2](#)
- [Figura 32: Estado túnel WireGuard - EC2](#)
- [Figura 33: Instancia WireGuard - OPNsense](#)
- [Figura 34: Conexión WireGuard - OPNsense](#)
- [Figura 35: Regla conexión red WG con DMZ - OPNsense](#)
- [Figura 36: Estado túnel WireGuard - OPNsense](#)
- [Figura 37: Agente registrado en SIEM DMZ - EC2](#)
- [Figura 38: Alertas SIEM - EC2](#)

d) Instalaciones

Proxmox

Guía oficial de instalación de Proxmox:
<https://pve.proxmox.com/pve-docs/chapter-pve-installation.html>

OPNsense

Guía oficial de instalación de OPNsense:
<https://docs.opnsense.org/manual/install.html>

Debian 13

Guía oficial de instalación de Debian 13:
<https://www.debian.org/releases/trixie/amd64/index>

Trapster

Repositorio del honeypot Trapster Community:
<https://github.com/0xBallpoint/trapster-community>

Wazuh

Guía oficial de instalación de Wazuh todo en uno:
<https://documentation.wazuh.com/current/quickstart.html>

Script todo en uno (v4.14):

<https://packages.wazuh.com/4.14/wazuh-install.sh>

FreeIPA

Guía oficial de instalación de FreeIPA:

https://www.freeipa.org/page/Quick_Start_Guide

Linux Mint

Guía oficial de instalación de Linux Mint:

<https://linuxmint-installation-guide.readthedocs.io/en/latest/install.html>

e) Código fuente

1. Modificaciones en Honeypot

A continuación se muestran los fragmentos de código modificados en el repositorio de Trapster para adaptarlo al proyecto.

1.1. docker-compose.yml

Archivo ubicado dentro del repositorio, trapster-community/**docker-compose.yml**.

Se añadió un volumen para persistir los logs del contenedor y mapearlos con la carpeta logs del repositorio:

```
volumes:  
  ./logs:/logs/  
  ./trapster/data:/app/trapster/data
```

1.2. logger.py

Archivo ubicado en la configuración de Trapster, trapster-community/trapster/**logger.py**.

Se modificó la generación de la marca del tiempo (timestamp) para usar el formato ISO 8601, necesario para que Wazuh pudiera interpretar correctamente los eventos. La línea modificada quedó de tal manera:

```
event = {  
    "timestamp":  
datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%S.%f")[:-3] + "+0000",  
    ...
```

```
}
```

1.3. ossec.conf

Archivo de configuración de Wazuh, ubicado en `/var/ossec/etc/ossec.conf`.

Se añadió un bloque **localfile** en la configuración del agente Wazuh para que monitorizara los logs generados por Trapster. La ruta apunta al archivo de log persistente gracias al volumen montado en el contenedor.

```
<ossec_config>
  <localfile>
    <log_format>json</log_format>
    <location>/home/cyberad/trapster-community/logs/trapster.log</location>
  </localfile>
</ossec_config>
```

2. Modificaciones en SIEM DMZ

2.1. local_rules.xml

```
<group name="trapster,honeypot">

<!-- ===== -->
<!-- SSH -->
<!-- ===== -->
<rule id="100005" level="7">
  <field name="logtype">ssh.connection</field>
  <description>SSH connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
    <id>T1021</id>
  </mitre>
</rule>

<rule id="100006" level="10">
  <field name="logtype">ssh.login</field>
  <description>SSH login attempt on honeypot</description>
  <mitre>
    <id>T1021</id>
    <id>T1110</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- FTP -->
<!-- ===== -->
<rule id="100010" level="7">
  <field name="logtype">ftp.connection</field>
  <description>FTP connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
  </mitre>
</rule>
```

```

<rule id="100011" level="10">
  <field name="logtype">ftp.login</field>
  <description>FTP login attempt on honeypot</description>
  <mitre>
    <id>T1021</id>
    <id>T1110</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- RDP -->
<!-- ===== -->
<rule id="100020" level="7">
  <field name="logtype">rdp.connection</field>
  <description>RDP connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
    <id>T1021</id>
  </mitre>
</rule>

<rule id="100021" level="10">
  <field name="logtype">rdp.login</field>
  <description>RDP login attempt on honeypot</description>
  <mitre>
    <id>T1021</id>
    <id>T1110</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- HTTP -->
<!-- ===== -->
<rule id="100030" level="7">
  <field name="logtype">http.query</field>
  <description>HTTP request to honeypot web service</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
  </mitre>

```

```

    </mitre>
</rule>

<!-- ===== -->
<!-- HTTPS -->
<!-- ===== -->
<rule id="100032" level="7">
  <field name="logtype">https.connection</field>
  <description>HTTPS connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
  </mitre>
</rule>

<rule id="100033" level="10">
  <field name="logtype">https.login</field>
  <description>HTTPS login attempt on honeypot</description>
  <mitre>
    <id>T1021</id>
    <id>T1110</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- TELNET -->
<!-- ===== -->
<rule id="100040" level="7">
  <field name="logtype">telnet.connection</field>
  <description>Telnet connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
    <id>T1021</id>
  </mitre>
</rule>

<rule id="100041" level="10">
  <field name="logtype">telnet.login</field>
  <description>Telnet login attempt on honeypot</description>
  <mitre>

```

```

    <id>T1021</id>
    <id>T1110</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- LDAP -->
<!-- ===== -->
<rule id="100050" level="7">
  <field name="logtype">ldap.connection</field>
  <description>LDAP connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
  </mitre>
</rule>

<rule id="100051" level="8">
  <field name="logtype">ldap.query</field>
  <description>LDAP enumeration attempt</description>
  <mitre>
    <id>T1087</id>
    <id>T1018</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- MySQL -->
<!-- ===== -->
<rule id="100060" level="7">
  <field name="logtype">mysql.connection</field>
  <description>MySQL connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
    <id>T1021</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- MSSQL -->

```



```

<!-- ===== -->
<rule id="100070" level="7">
  <field name="logtype">mssql.connection</field>
  <description>MSSQL connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
    <id>T1021</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- VNC -->
<!-- ===== -->
<rule id="100080" level="7">
  <field name="logtype">vnc.connection</field>
  <description>VNC connection attempt to honeypot</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
    <id>T1021</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- DNS -->
<!-- ===== -->
<rule id="100090" level="6">
  <field name="logtype">dns.query</field>
  <description>DNS query to honeypot</description>
  <mitre>
    <id>T1046</id>
    <id>T1590</id>
    <id>T1595</id>
  </mitre>
</rule>

<!-- ===== -->
<!-- RSYNC -->
<!-- ===== -->
<rule id="100100" level="7">

```

```

<field name="logtype">rsync.connection</field>
<description>Rsync connection attempt to honeypot</description>
<mitre>
  <id>T1595</id>
  <id>T1046</id>
</mitre>
</rule>

<!-- ===== -->
<!-- BRUTE FORCE GLOBAL -->
<!-- ===== -->
<rule id="100999" level="12" frequency="5" timeframe="10">
  <if_matched_group>trapster</if_matched_group>
  <same_field>src_ip</same_field>
  <description>Multiple honeypot attacks from same IP</description>
  <mitre>
    <id>T1595</id>
    <id>T1110</id>
    <id>T1021</id>
  </mitre>
</rule>

</group>

<!-- ===== -->
<!-- EC2 IPTABLES LOGGING -->
<!-- ===== -->

<group name="attack,iptables,">

  <rule id="140010" level="6">
    <if_sid>4100</if_sid>
    <match>IPTables-Attack-Log</match>
    <description>BASE: External connection attempt detected on EC2 from
$(srcip)</description>
    <mitre>
      <id>T1595</id>
      <id>T1046</id>
    </mitre>
  </rule>

```

```
<rule id="140015" level="8" frequency="2" timeframe="2">
  <if_matched_sid>140010</if_matched_sid>
  <same_source_ip />
  <description>External connection attempt detected on EC2 from
$(srcip)</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
  </mitre>
</rule>

<rule id="140030" level="12" frequency="5" timeframe="30">
  <if_matched_sid>140015</if_matched_sid>
  <same_source_ip />
  <description>Repeated external probing detected from $(srcip)</description>
  <mitre>
    <id>T1595</id>
    <id>T1046</id>
    <id>T1110</id>
  </mitre>
</rule>

</group>
```

3. Modificaciones en EC2

3.1. wg0.conf

Archivo de configuración de WireGuard, ubicado en `/etc/wireguard/wg0.conf`.

```
[Interface]
PrivateKey = LLAVE_PRIVADA_GENERADA=
Address = 10.10.0.1/24
ListenPort = 65534

PostUp = iptables -t nat -A PREROUTING -p tcp --dport 1:20207 -j DNAT
--to-destination 172.26.10.100
PostUp = iptables -t nat -A PREROUTING -p tcp --dport 20209:65500 -j DNAT
--to-destination 172.26.10.100
PostUp = iptables -A FORWARD -p tcp -d 172.26.10.100 -j ACCEPT
PostUp = iptables -t nat -A POSTROUTING -o wg0 -j MASQUERADE

PostDown = iptables -t nat -D PREROUTING -p tcp --dport 1:20207 -j DNAT
--to-destination 172.26.10.100 2>/dev/null || true
PostDown = iptables -t nat -D PREROUTING -p tcp --dport 20209:65500 -j DNAT
--to-destination 172.26.10.100 2>/dev/null || true
PostDown = iptables -D FORWARD -p tcp -d 172.26.10.100 -j ACCEPT 2>/dev/null ||
true
PostDown = iptables -t nat -D POSTROUTING -o wg0 -j MASQUERADE 2>/dev/null ||
true

[Peer]
PublicKey = LLAVE_PUBLICA_DELA_CONEXION=
PresharedKey = LLAVE_COMPARTIDA=
AllowedIPs = 10.10.0.2/32, 172.26.10.0/24
PersistentKeepalive = 15
```

3.2. ossec.conf

Archivo de configuración del agente de wazuh, ubicado en **/var/ossec/etc/ossec.conf**, se añadió un bloque localfile para la recolección de logs de iptables con las direcciones IP públicas.

```
<ossec_config>
  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/kern.log</location>
  </localfile>
</ossec_config>
```